

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_eb99bd9f-440\agent-abf6521ed27c075a1.jsonl`  
- SHA-256: `6ebb2b7909a7b0b830881933126d600dc087f98d2e7fd072ccdfcdf2a32a4bc3`  
- Source modified: `2026-07-06T06:09:54+00:00`  
- Imported at: `2026-07-08T16:00:13+00:00`  
- project: `wf_eb99bd9f-440`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T06:02:53.383Z)

You are reviewing GitHub PR #35 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-b-domain-schema -> main).
It implements ADR 0011 (docs/adr/0011-provider-neutral-domain-schema.md): a provider-neutral domain schema (Track B). Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/models.py, tests/test_db.py, tests/test_delivery.py. Diff: +1299/-150.

Ground rules:

- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run the venv python for a targeted repro:
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. The DB layer is aiosqlite; use db.__apply_schema(conn) on an aiosqlite ":memory:" connection to build the schema, and db.upsert_account(...) to seed. A working repro is stronger evidence than prose.
- Compare against v1 behaviour on main when relevant: git -C C:/Users/avidu/Projects/Telegram show main:src/tg_video_filter/db.py
- Authoritative local gate ALREADY RUN (do not re-run the full suite): ruff check clean; pytest 434 passed @ 96.63% coverage (floor 80%); ruff format --check FAILS only on commands.py which is NOT in this PR (pre-existing on main) ? do not report that as a PR finding.
- Report ONLY defects that ship in THIS PR's diff. No style nits already passing ruff. No praise.
- Each finding needs a concrete failure scenario (inputs -> wrong result). Rank most-severe first.
- Severity: blocker (data loss/corruption/wrong core behaviour), high (real bug, narrow trigger), medium (latent/edge), low (hygiene). Be honest; downgrade if trigger is unrealistic.

DIMENSION: conformance to the ACCEPTED ADR 0011 spec (read the ADR fully).

Investigate specifically:

- Does the implemented schema match every column/constraint in the ADR's proposed schema (sources, content_items, policies, destinations, routing_rules, delivery_attempts)? Note any divergence.
 - ADR claims Option 1A "generalizes ADR 0006 without losing any dedup quality." Given the Tier-2 finding, is that claim actually TRUE as implemented? If the implementation faithfully implements the ADR but the ADR's no-loss claim is itself wrong, say so explicitly (design-level finding: the accepted design drops a dedup tier the README still advertises).
 - Deprecated-table retention, is_default single-default enforcement (ADR says UNIQUE partial idx; is it actually created?), watched=1 backfill, empty Track-C tables present.
- Return findings per schema. Divergences from the accepted spec, and false claims in ADR/PR text,

are findings.

2. assistant (2026-07-06T06:02:58.173Z)

I'll start by reading the ADR spec and the changed files.

3. user (2026-07-06T06:02:59.159Z)

```
1      # ADR 0011: Provider-neutral domain schema (Track B)
2
3      - **Status**: Accepted (Question 1 ? Option 1A; Question 2 ? Option 2A; PR #34)
4      - **Date**: 2026-07-06
5      - **Decision owner**: project owner
6      - **Tracking issue**: #27 (Track B of the provider-neutral umbrella, #25)
7      - **Depends on**: ADR 0010 (Track A ? `accounts` + `telegram_sessions`, merged in #33)
8      - **Blocks**: #28 (Track C ? delivery/routing), #29 (Track D ? commands), and indirectly
#30 (Track E)
9
10     ## Context
11
12     ADR 0010 split `users` into `accounts` (product identity) + `telegram_sessions`
13     (MTPProto credential). That unblocked the identity dimension. This ADR
14     tackles the **content dimension**: how to represent sources, content items,
15     policies, destinations, routes, and delivery attempts in a provider-neutral
16     way, so that the system can ingest from Telegram today and from WhatsApp
17     (or any future provider) later without another schema rewrite.
18
19     ### What we have today (post-Track-A)
20
21     | Table | Meaning | Provider-coupling |
22     |---|---|---|
23     | `accounts` | Product identity (ADR 0010) | provider-neutral (`provider` column) |
24     | `telegram_sessions` | Optional MTPProto session (ADR 0010) | Telegram-specific by
design |
25     | `channels` | A Telegram channel/chat an account is a member of | **Telegram-specific**
(`channel_id` is a Telegram id) |
26     | `videos` | A seen Telegram video message + metadata | **Telegram-specific**
(`message_id`, `document_id` are Telegram ids) |
27     | `filter_config` | Per-account JSON blob of `FilterConfig` | provider-neutral in shape,
but scoped per-account not per-source |
28
29     The dedup logic (ADR 0006) is **3-tier and Telegram-specific**:
30     - Tier 0: `(user_id, channel_id, message_id)` ? same Telegram message.
31     - Tier 1: `(user_id, document_id)` ? same Telegram file forwarded across channels.
32     - Tier 2: `(user_id, size_bytes, duration_s, height, width)` ? re-upload fingerprint.
33
34     ### What we need
35
36     The umbrella (#25) introduces six concepts: `Source`, `ContentItem`, `Policy`,
37     `Destination`, `Route` (`routing_rules`), and `DeliveryAttempt`. This ADR
38     defines the schema for each, the **two open design questions** that gate the
39     schema, and the migration path from the current tables.
40
41     ## The two decisions I need from you
42
43     ### Question 1: `ContentItem.dedup_key` ? how does provider-neutral dedup relate to ADR
0006?
44
45     The current 3-tier dedup is Telegram-specific (`document_id` is a Telegram
46     concept). A provider-neutral `ContentItem` needs a dedup story that works
47     across providers. Three options:
48
49     ##### Option 1A (recommended): tiered lookup that wraps ADR 0006's tiers
50
51     Keep ADR 0006's three tiers but generalize them behind a single
```

```

52 `dedup_key` column on `content_items`:
53
54 ```
55 content_items.dedup_key TEXT NOT NULL
56 ```
57
58 The *value* of `dedup_key` is computed by an explicit decision tree at
59 ingestion time, provider by provider. The tree is ordered strongest-key
60 first and each branch is **guarded so partial metadata can never collapse
61 to a degenerate key** (e.g. `tg:fp:0:0:0:0`):
62
63 - **Telegram:**
64   1. If `document_id` is present ? `dedup_key` = "tg:doc:{document_id}".
65   2. Else if **all four** fingerprint fields (`size_bytes`, `duration_s`,
66       `height`, `width`) are present ? `dedup_key` = "tg:fp:{size}:{dur}:{h}:{w}".
67   3. Else ? `dedup_key` = "tg:msg:{source_id}:{provider_message_id}"
68       (Tier 0; the source/channel id is required because Telegram
69       `message_id` is only unique within a channel, not across sources).
70 - **Future WhatsApp:** `dedup_key` = "wa:msg:{message_id}" (WhatsApp message
71     ids are globally unique per account) or "wa:sha256:{blob_hash}" if WA
72     exposes a content hash.
73
74 The guard in step 2 (all four fields required) is what prevents the
75 partial-metadata collision the first draft of this ADR allowed. The pure
76 `compute_dedup_key(provider, *, document_id, source_id, provider_message_id,
77 size_bytes, duration_s, height, width)` function that implements this tree
78 gets its own unit tests in the implementation PR, including cases for
79 partial/null metadata.
80
81 A **partial unique index** on `(account_id, dedup_key) WHERE dedup_key IS NOT NULL`
82 replaces the current Tier-1 index and enforces per-account dedup at the DB
83 level, regardless of provider.
84
85 **Why recommended:**
86 - Zero new bandwidth ? all keys are computed from metadata already extracted
87   (honors ADR 0003's no-download principle).
88 - The provider prefix (`tg:` / `wa:`) makes cross-provider collisions
89   structurally impossible.
90 - Existing `videos` rows migrate cleanly: `dedup_key` is computed from their
91   existing `document_id` / metadata columns during the migration.
92 - Supersedes ADR 0006 cleanly: the three tiers become *one* column with
93   provider-specific derivation rules, documented in the migration.
94
95 **Costs:**
96 - The migration must compute `dedup_key` for every existing `videos` row
97   (cheap ? pure SQL string building, no I/O).
98 - The `dedup_key` derivation logic lives in code (a pure function per
99   provider), not in the schema. This is a feature, but it means the
100   derivation rules need their own unit tests.
101
102 ##### Option 1B: content hash (SHA-256 of downloaded bytes)
103
104 `dedup_key` = sha256(downloaded_content)`. Cryptographically strong,
105 provider-agnostic, catches re-encodes that 1A misses.
106
107 **Rejected** ? violates ADR 0003 (metadata-first, no download in the hot
108 path). Downloading every incoming video to hash it would multiply bandwidth
109 and latency by orders of magnitude. Could be added later as an optional
110 deep-inspect stage (deferred to Track F per #31).
111
112 ##### Option 1C: provider-assigned only (drop Tier 2)
113
114 `dedup_key` = "tg:doc:{document_id}" always; no metadata fingerprint
115 fallback. Simpler, but loses Tier-2 dedup for re-uploads that get a new
116 `document.id` ? a real and common case (someone re-encodes and re-uploads

```

```

117     the same video to a different channel).
118
119     **Recommendation: 1A.** It preserves the dedup quality of ADR 0006, generalizes cleanly,
and honors the no-download constraint.
120
121     ---
122
123     ### Question 2: `Policy` ? existing `filter_config` ? rename, or wrap?
124
125     The codebase already has `filter_config.config_json` (one row per account,
126     holding a `FilterConfig` JSON blob ? see `models.py` and ADR 0005). The
127     umbrella introduces a `Policy` concept. They overlap. Two options:
128
129     #### Option 2A (recommended): `Policy` *generalizes* `filter_config` (rename + extend)
130
131     `policies` is a new table; `filter_config` is migrated into it. One account
132     can own multiple policies (enables per-route filter overrides from ADR
133     0009), with a `DEFAULT` policy per account for the no-rules-configured case.
134
135     ```
136     policies (
137         id            INTEGER PRIMARY KEY AUTOINCREMENT,
138         account_id    INTEGER NOT NULL REFERENCES accounts(id),
139         name          TEXT NOT NULL,                -- e.g. "default", "reggae-route"
140         config_json   TEXT NOT NULL,                -- the FilterConfig blob (Phase 1)
141         is_default    INTEGER NOT NULL DEFAULT 0,    -- one per account (UNIQUE partial
idx)
142         created_at    TEXT NOT NULL,
143         updated_at    TEXT NOT NULL,
144         UNIQUE (account_id, name)
145     );
146     ```
147
148     **Phase 1** (this track): `config_json` holds exactly today's `FilterConfig`
149     JSON ? the rename is purely structural. **Phase 2** (Track F, deferred):
150     `config_json` grows to include topic preferences and security rules, but
151     that's behind the same column so no further migration.
152
153     Migration: for each `filter_config` row, insert a `policies` row with
154     `name='default'`, `is_default=1`, `config_json=<the existing blob>`. Drop
155     `filter_config` after the migration (keep `filter_config_deprecated` for one
156     release, mirroring ADR 0010's safety-net pattern).
157
158     **Why recommended:**
159     - `Policy` becomes the single home for "what to filter/enrich" ? no second
160     overlapping concept. Today's `FilterConfig` is just the Phase-1 shape.
161     - Enables ADR 0009's per-route filter overrides (`routing_rules.policy_id`)
162     without a second migration later.
163     - The existing `db.load_filter_config(account_id)` / `save_filter_config`
164     become thin shims over `policies WHERE account_id=? AND is_default=1`,
165     mirroring the Track-A `User` shim pattern ? backwards-compatible.
166
167     **Costs:**
168     - One more table migration. `filter_config` ? `policies` is a rename+extend.
169     - The `FilterConfig` Pydantic model stays (it's the Phase-1 config shape);
170     `Policy` is the *container* (name, owner, default flag) around it. Two
171     models, not one ? but they compose cleanly.
172
173     #### Option 2B: keep `filter_config` as-is; `Policy` is a wrapping layer
174
175     Don't touch `filter_config`. `policies` references a `filter_config` row
176     plus adds enrichment/security stages as separate columns.
177
178     **Rejected** ? creates two overlapping concepts (`filter_config` for the
179     v1 rules, `policies` for "everything else"), which is exactly the

```

```

180 conflation the umbrella exists to dissolve. We'd be back here in a year
181 merging them.
182
183 **Recommendation: 2A.** One concept, one table, phased growth via the same JSON column.
184
185 ---
186
187 ## Proposed schema (assuming 1A + 2A)
188
189 ### `sources` (generalizes `channels`)
190
191 A configured place we ingest from. Maps 1:1 to today's per-account
192 `channels` rows for Telegram; future providers add their own source kinds.
193
194 ```
195 sources (
196     id                INTEGER PRIMARY KEY AUTOINCREMENT,
197     account_id        INTEGER NOT NULL REFERENCES accounts(id),
198     provider          TEXT NOT NULL DEFAULT 'telegram',
199     provider_source_id TEXT NOT NULL,          -- Telegram channel_id; future: WA group id
200     title             TEXT,
201     kind              TEXT NOT NULL DEFAULT 'channel', -- 'channel' | 'group' | 'feed'
202     watched           INTEGER NOT NULL DEFAULT 0,      -- ADR 0009 selective scanning
203     seen_first_at     TEXT NOT NULL,
204     last_seen_at      TEXT,
205     UNIQUE (account_id, provider, provider_source_id)
206 );
207 ```
208
209 `channels.user_id` (now ? `accounts.id`) maps to `sources.account_id`;
210 `channels.channel_id` maps to `sources.provider_source_id`. The `watched`
211 flag (ADR 0009, deferred to Track E) is included here so Track E doesn't
212 need another migration.
213
214 ### `content_items` (generalizes `videos`)
215
216 A normalized item discovered from any source. Phase 1 stores only videos
217 (matches today's `videos`); the columns are provider-neutral but the
218 Phase-1 migration only populates video fields.
219
220 ```
221 content_items (
222     id                INTEGER PRIMARY KEY AUTOINCREMENT,
223     account_id        INTEGER NOT NULL REFERENCES accounts(id),
224     source_id         INTEGER NOT NULL REFERENCES sources(id),
225     provider          TEXT NOT NULL DEFAULT 'telegram',
226     provider_message_id TEXT NOT NULL,          -- Telegram message_id; future: WA message
227     content_type       TEXT NOT NULL DEFAULT 'video', -- 'video' | 'link' | 'article' |
228     ...
229     text              TEXT,                    -- caption / post text (nullable)
230     link_url          TEXT,                    -- for link/article posts
231     media_ref         TEXT,                    -- provider media reference (file path,
232     blob id)
233     document_id       TEXT,                    -- Telegram document.id (kept for TG-
234     specific lookups)
235     duration_s        INTEGER,
236     width             INTEGER,
237     height            INTEGER,
238     size_bytes        INTEGER,
239     mime_type         TEXT,
240     dedup_key         TEXT NOT NULL,          -- ADR 0011 Question 1 (Option 1A)
241     matched           INTEGER NOT NULL DEFAULT 0,
242     seen_at           TEXT NOT NULL,
243     -- Tier-0 equivalent: provider_message_id is only unique within a source

```

```

241         -- (a Telegram message_id is per-channel), so source_id is required to
242         -- avoid false collisions across sources for the same account.
243         UNIQUE (account_id, source_id, provider_message_id),
244         UNIQUE (account_id, dedup_key)                                -- Tier-1/2 equivalent (Option
1A)
245     );
246     ```
247
248     **Note on `forwarded`:** dropped from `content_items`. Per-destination
249     delivery state moves to `delivery_attempts` (Track C, #28). For Phase 1
250     (before Track C lands), `content_items.matched` alone is sufficient ?
251     delivery still goes to the single `delivery_target`, and the boolean
252     `forwarded` is retained temporarily on a *deprecated* `videos`-compat view
253     so the v1 delivery path keeps working. Track C removes it.
254
255     ### `policies` (generalizes `filter_config`) ? Option 2A
256
257     As specified in Question 2 above.
258
259     ### `destinations` (new ? Track C will populate)
260
261     Defined here so the schema is complete, but **Track C (#28) implements the
262     delivery layer**. Phase-1 presence is the table only (no delivery logic).
263
264     ```
265     destinations (
266         id                INTEGER PRIMARY KEY AUTOINCREMENT,
267         account_id         INTEGER NOT NULL REFERENCES accounts(id),
268         provider           TEXT NOT NULL DEFAULT 'telegram',
269         target             TEXT NOT NULL,                -- 'me', '@username', chat id (ADR 0005)
270         kind               TEXT NOT NULL DEFAULT 'saved', -- 'saved' | 'channel' | 'chat'
271         title              TEXT,
272         is_output          INTEGER NOT NULL DEFAULT 0,    -- ADR 0009: distinguishes output
channels
273         created_at        TEXT NOT NULL,
274         UNIQUE (account_id, provider, target)
275     );
276     ```
277
278     ### `routing_rules` (new ? Track C will populate)
279
280     Per ADR 0009 (canonical name locked in issue #25).
281
282     ```
283     routing_rules (
284         id                INTEGER PRIMARY KEY AUTOINCREMENT,
285         account_id         INTEGER NOT NULL REFERENCES accounts(id),
286         source_id          INTEGER NOT NULL REFERENCES sources(id),
287         policy_id          INTEGER,                      -- nullable ? use account's default policy
288         destination_id     INTEGER NOT NULL REFERENCES destinations(id),
289         enabled            INTEGER NOT NULL DEFAULT 1,
290         priority           INTEGER NOT NULL DEFAULT 0,
291         created_at         TEXT NOT NULL,
292         UNIQUE (account_id, source_id, destination_id)
293     );
294     ```
295
296     ### `delivery_attempts` (new ? Track C will populate)
297
298     Per #28's idempotency requirement (unique index on
299     `(content_item_id, destination_id)`).
300
301     ```
302     delivery_attempts (
303         id                INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

304     content_item_id INTEGER NOT NULL REFERENCES content_items(id),
305     destination_id  INTEGER NOT NULL REFERENCES destinations(id),
306     route_id        INTEGER REFERENCES routing_rules(id),
307     status           TEXT NOT NULL DEFAULT 'pending', --
'pending' | 'delivered' | 'failed' | 'retrying'
308     attempt_count   INTEGER NOT NULL DEFAULT 0,
309     last_error       TEXT,
310     created_at       TEXT NOT NULL,
311     delivered_at     TEXT,
312     next_retry_at    TEXT,
313     UNIQUE (content_item_id, destination_id)          -- ADR #28 idempotency
314 );
315 ```
316
317 ## Migration strategy (integer-preserving, phased)
318
319 This track lands sources, content_items, and policies only.
320 destinations, routing_rules, delivery_attempts are defined in the
321 schema (so the ADR is complete) but their tables are created empty and
322 populated by Track C. This keeps Track B focused and reviewable.
323
324 ### Migration _migration_003_domain_schema (idempotent)
325
326 1. Create sources, content_items, policies, destinations,
327 routing_rules, delivery_attempts (all IF NOT EXISTS).
328 2. Backfill sources from channels:
329 provider='telegram', provider_source_id=channel_id, watched=1
330 (ADR 0009 backfill: already-seen channels stay watched).
331 3. Backfill content_items from videos, computing dedup_key via the
332 explicit decision tree from Option 1A (every branch reachable, no
333 COALESCE(...,0) collapsing partial metadata to a degenerate key):
334 - If document_id IS NOT NULL:
335 dedup_key = 'tg:doc:' || document_id.
336 - Else if size_bytes IS NOT NULL AND duration_s IS NOT NULL AND height IS NOT NULL
AND width IS NOT NULL:
337 dedup_key = 'tg:fp:' || size_bytes || ':' || duration_s || ':' || height || ':' ||
width.
338 - Else (Tier-0 fallback; partial or missing media metadata):
339 dedup_key = 'tg:msg:' || source_id || ':' || provider_message_id,
340 where source_id is the joined sources.id for the row's legacy
341 channel_id and provider_message_id is the legacy message_id.
342
343 The implementation PR must include migration tests that seed rows hitting
344 each branch (including all-fingerprint-fields-null) and assert no two
345 rows collapse to the same dedup_key.
346 4. Backfill policies from filter_config: one row per account,
347 name='default', is_default=1, config_json copied verbatim.
348 5. Rename (don't drop) channels ? channels_deprecated,
349 videos ? videos_deprecated, filter_config ? filter_config_deprecated.
350 Kept for one release as a rollback safety net (ADR 0010 pattern).
351
352 The legacy videos/channels/filter_config tables and their
353 db.load_filter_config / upsert_channel / insert_video repos stay
354 working as shims over the new tables (same pattern as the Track-A User
355 shim), so existing call sites in telethon_client.py, backfill.py, and
356 commands.py keep working without a flag day.
357
358 ## Consequences
359
360 Positive
361 - Provider-neutral schema: sources, content_items, policies,
362 destinations, routing_rules, delivery_attempts all carry a
363 provider column or provider-prefixed keys. Adding WhatsApp later is a
364 new provider='whatsapp' value, not a new schema.
365 - dedup_key (Option 1A) generalizes ADR 0006 without losing any dedup

```

```

366     quality and without downloading anything.
367 - `Policy` (Option 2A) gives a single home for filter + future enrichment
368     config, enabling ADR 0009's per-route overrides with no second migration.
369 - Backwards-compatible via shims, mirroring the Track-A pattern that
370     already passed review.
371
372 **Negative**
373 - Six new tables; the schema grows significantly. Each is small and
374     single-purpose, but the surface area is larger.
375 - The dedup-key derivation logic is in code, not the schema ? needs its
376     own unit tests (a `compute_dedup_key(provider, **metadata)` pure function).
377 - Phase 1 carries some dead-on-arrival columns (`content_items.link_url`,
378     `text`) that aren't populated until later tracks. This is intentional
379     (avoid future migrations) but means some columns are NULL until then.
380
381 ## Alternatives considered
382
383 - **Do nothing; keep `channels`/`videos`/`filter_config` and special-case
384     providers in code.** Rejected ? same conflation argument as ADR 0010.
385     Telegram-specific ids in column names (`channel_id`, `message_id`,
386     `document_id`) would leak into every future provider's code path.
387 - **Land all six tables' *logic* in this track.** Rejected ? Track B is
388     already the largest track. Splitting schema (Track B) from delivery
389     logic (Track C) keeps each PR reviewable. The empty `destinations` /
390     `routing_rules` / `delivery_attempts` tables in this track are the
391     contract Track C codes against.
392 - **`content_items` as a view over `videos` (no migration).** Rejected ?
393     views don't give us the `provider` column or the `dedup_key` index, and
394     SQLite view support for writes is limited. A real table is needed.
395
396 ## Open questions for the decision owner
397
398 1. **Question 1** ? `dedup_key`: 1A (tiered, recommended), 1B (hash,
399     rejected), or 1C (provider-assigned only).
400 2. **Question 2** ? `Policy` ? `filter_config`: 2A (generalize, recommended)
401     or 2B (wrap, rejected).
402 3. **Dead columns in Phase 1** ? acceptable to land `content_items.link_url`
403     / `text` as NULL until later tracks populate them? Recommendation: yes
404     (avoids a future `ALTER TABLE`).
405 4. **Deprecated-table retention** ? one release (default), matching ADR 0010?
406     Recommendation: yes.
407
408 ## References
409
410 - ADR 0003 ? metadata-first probing (the no-download constraint that rules
411     out Option 1B's content hashing).
412 - ADR 0005 ? per-user delivery target (generalized by `destinations` /
413     `routing_rules`).
414 - ADR 0006 ? content dedup (generalized by `content_items.dedup_key`,
415     Option 1A).
416 - ADR 0009 ? selective scanning + multiplexed routing (depends on
417     `sources.watched` and `routing_rules`, both defined here).
418 - ADR 0010 ? identity/session separation (this ADR's `account_id` FKs point
419     at `accounts.id`).
420 - Issue #27 ? Track B (this ADR's tracking issue).
421 - Issue #25 ? umbrella; defines the six concepts and the implementation order.
422

```

4. user (2026-07-06T06:02:59.942Z)

```

1     """SQLite persistence layer (aiosqlite).
2
3     Schema, migrations, and repository functions. See ``docs/design.md`` §3 for
4     the legacy data model and ``docs/adr/0011-provider-neutral-domain-schema.md``
5     for the provider-neutral ``sources`` / ``content_items`` / ``policies`` /

```



```

6     ``destinations`` / ``routing_rules`` / ``delivery_attempts`` schema (ADR 0011).
7
8     The repository functions take an ``aiosqlite.Connection`` so tests can inject an
9     in-memory database (``aiosqlite.connect(":memory:")``).
10    """
11
12    from __future__ import annotations
13
14    from collections.abc import Awaitable, Callable, Iterable
15    from datetime import UTC, datetime
16
17    import aiosqlite
18
19    from tg_video_filter.models import (
20        Account,
21        Channel,
22        ContentItem,
23        FilterConfig,
24        Policy,
25        Source,
26        TelegramSession,
27        User,
28        VideoRecord,
29    )
30
31    # --- DDL -----
32
33    _SCHEMA = """
34    CREATE TABLE IF NOT EXISTS accounts (
35        id            INTEGER PRIMARY KEY,
36        telegram_id   INTEGER UNIQUE,
37        provider      TEXT NOT NULL DEFAULT 'telegram',
38        display_name  TEXT,
39        created_at    TEXT NOT NULL
40    );
41
42    CREATE TABLE IF NOT EXISTS telegram_sessions (
43        account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
44        phone         TEXT,
45        session_path  TEXT NOT NULL,
46        enabled       INTEGER NOT NULL DEFAULT 1,
47        created_at    TEXT NOT NULL,
48        last_login_at TEXT
49    );
50
51    CREATE TABLE IF NOT EXISTS channels (
52        id            INTEGER PRIMARY KEY AUTOINCREMENT,
53        user_id       INTEGER NOT NULL REFERENCES accounts(id),
54        channel_id    INTEGER NOT NULL,
55        title         TEXT,
56        seen_first_at TEXT NOT NULL,
57        UNIQUE (user_id, channel_id)
58    );
59
60    CREATE TABLE IF NOT EXISTS videos (
61        id            INTEGER PRIMARY KEY AUTOINCREMENT,
62        user_id       INTEGER NOT NULL REFERENCES accounts(id),
63        channel_id    INTEGER NOT NULL,
64        message_id    INTEGER NOT NULL,
65        document_id   INTEGER,
66        duration_s    INTEGER,
67        width         INTEGER,
68        height        INTEGER,
69        size_bytes    INTEGER,
70        mime_type     TEXT,

```

```

71         matched      INTEGER NOT NULL DEFAULT 0,
72         forwarded     INTEGER NOT NULL DEFAULT 0,
73         seen_at       TEXT NOT NULL,
74         UNIQUE (user_id, channel_id, message_id)
75     );
76
77     CREATE TABLE IF NOT EXISTS filter_config (
78         user_id        INTEGER PRIMARY KEY REFERENCES accounts(id),
79         config_json    TEXT NOT NULL
80     );
81
82     -- Provider-neutral domain tables (ADR 0011, Track B). ``sources``,
83     -- ``content_items``, and ``policies`` are populated by migration 003 (and by
84     -- the shims below); ``destinations`` / ``routing_rules`` / ``delivery_attempts``
85     -- are created empty here so the schema is complete ? Track C (#28) implements
86     -- the delivery logic that populates them.
87
88     CREATE TABLE IF NOT EXISTS sources (
89         id              INTEGER PRIMARY KEY AUTOINCREMENT,
90         account_id      INTEGER NOT NULL REFERENCES accounts(id),
91         provider        TEXT NOT NULL DEFAULT 'telegram',
92         provider_source_id TEXT NOT NULL,
93         title           TEXT,
94         kind            TEXT NOT NULL DEFAULT 'channel',
95         watched         INTEGER NOT NULL DEFAULT 0,
96         seen_first_at   TEXT NOT NULL,
97         last_seen_at    TEXT,
98         UNIQUE (account_id, provider, provider_source_id)
99     );
100
101     CREATE TABLE IF NOT EXISTS content_items (
102         id              INTEGER PRIMARY KEY AUTOINCREMENT,
103         account_id      INTEGER NOT NULL REFERENCES accounts(id),
104         source_id       INTEGER NOT NULL REFERENCES sources(id),
105         provider        TEXT NOT NULL DEFAULT 'telegram',
106         provider_message_id TEXT NOT NULL,
107         content_type    TEXT NOT NULL DEFAULT 'video',
108         text            TEXT,
109         link_url        TEXT,
110         media_ref       TEXT,
111         document_id     TEXT,
112         duration_s      INTEGER,
113         width           INTEGER,
114         height          INTEGER,
115         size_bytes      INTEGER,
116         mime_type       TEXT,
117         dedup_key       TEXT NOT NULL,
118         matched         INTEGER NOT NULL DEFAULT 0,
119         seen_at         TEXT NOT NULL,
120         UNIQUE (account_id, source_id, provider_message_id),
121         UNIQUE (account_id, dedup_key)
122     );
123
124     CREATE TABLE IF NOT EXISTS policies (
125         id              INTEGER PRIMARY KEY AUTOINCREMENT,
126         account_id      INTEGER NOT NULL REFERENCES accounts(id),
127         name            TEXT NOT NULL,
128         config_json     TEXT NOT NULL,
129         is_default      INTEGER NOT NULL DEFAULT 0,
130         created_at      TEXT NOT NULL,
131         updated_at      TEXT NOT NULL,
132         UNIQUE (account_id, name)
133     );
134
135     CREATE TABLE IF NOT EXISTS destinations (

```

```

136         id            INTEGER PRIMARY KEY AUTOINCREMENT,
137         account_id    INTEGER NOT NULL REFERENCES accounts(id),
138         provider      TEXT NOT NULL DEFAULT 'telegram',
139         target        TEXT NOT NULL,
140         kind          TEXT NOT NULL DEFAULT 'saved',
141         title         TEXT,
142         is_output     INTEGER NOT NULL DEFAULT 0,
143         created_at    TEXT NOT NULL,
144         UNIQUE (account_id, provider, target)
145     );
146
147     CREATE TABLE IF NOT EXISTS routing_rules (
148         id            INTEGER PRIMARY KEY AUTOINCREMENT,
149         account_id    INTEGER NOT NULL REFERENCES accounts(id),
150         source_id     INTEGER NOT NULL REFERENCES sources(id),
151         policy_id     INTEGER REFERENCES policies(id),
152         destination_id INTEGER NOT NULL REFERENCES destinations(id),
153         enabled       INTEGER NOT NULL DEFAULT 1,
154         priority      INTEGER NOT NULL DEFAULT 0,
155         created_at    TEXT NOT NULL,
156         UNIQUE (account_id, source_id, destination_id)
157     );
158
159     CREATE TABLE IF NOT EXISTS delivery_attempts (
160         id            INTEGER PRIMARY KEY AUTOINCREMENT,
161         content_item_id INTEGER NOT NULL REFERENCES content_items(id),
162         destination_id INTEGER NOT NULL REFERENCES destinations(id),
163         route_id      INTEGER REFERENCES routing_rules(id),
164         status        TEXT NOT NULL DEFAULT 'pending',
165         attempt_count INTEGER NOT NULL DEFAULT 0,
166         last_error    TEXT,
167         created_at    TEXT NOT NULL,
168         delivered_at  TEXT,
169         next_retry_at TEXT,
170         UNIQUE (content_item_id, destination_id)
171     );
172     """
173
174     # Note: the ``idx_videos_user_document`` partial unique index is created by
175     # migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
176     # which is present on fresh tables but only added to legacy v1 DBs by the
177     # migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
178     # DBs that don't yet have the column.
179
180
181     async def init_db(db_path: str) -> None:
182         """Open the database at ``db_path`` and create/migrate the schema.
183
184         Safe to call on every startup. Applies the base schema (``IF NOT EXISTS``)
185         then runs any pending migrations via the ``user_version`` ladder.
186         """
187         async with aiosqlite.connect(db_path) as conn:
188             await _apply_schema(conn)
189
190
191     async def _apply_schema(conn: aiosqlite.Connection) -> None:
192         # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
193         # and a busy timeout so concurrent access waits instead of failing immediately.
194         await conn.execute("PRAGMA journal_mode = WAL")
195         await conn.execute("PRAGMA foreign_keys = ON")
196         await conn.execute("PRAGMA busy_timeout = 5000")
197         await conn.executescript(_SCHEMA)
198         # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
199         # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
200         # migration 002's FK rebuild and migration 003's backfill on legacy DBs).

```

```

201     # Once migration 003 has run (signalled by ``channels_deprecated`` existing),
202     # any *empty* recreations of those legacy tables from a subsequent
203     # ``init_db`` are dropped here so they don't shadow the renamed copies.
204     if await _table_exists(conn, "channels_deprecated"):
205         await conn.execute("DROP TABLE IF EXISTS channels")
206         await conn.execute("DROP TABLE IF EXISTS videos")
207         await conn.execute("DROP TABLE IF EXISTS filter_config")
208     await _apply_migrations(conn)
209     await conn.commit()
210
211
212     # --- migrations -----
213     #
214     # Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
215     # Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
216     # the source of truth for *fresh* databases; the migrations below reconcile
217     # *existing* databases that were created by an older schema version.
218     #
219     # To add a new migration:
220     # 1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
221     # 2. Implement ``async def _migration_NN(conn) -> None`` (must be idempotent).
222     # 3. Add a test in ``tests/test_db.py`` that seeds an old schema, runs
223     #    ``init_db``, and asserts the new structure.
224
225     _MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
226
227
228     def _migration(
229         version: int,
230     ) -> Callable[
231         [Callable[[aiosqlite.Connection], Awaitable[None]]],
232         Callable[[aiosqlite.Connection], Awaitable[None]],
233     ]:
234         """Register a migration that brings the DB up to ``user_version == version``."""
235
236         def decorator(
237             fn: Callable[[aiosqlite.Connection], Awaitable[None]],
238         ) -> Callable[[aiosqlite.Connection], Awaitable[None]]:
239             _MIGRATIONS.append((version, fn))
240             return fn
241
242         return decorator
243
244
245     # --- provider-neutral dedup key (ADR 0011, Option 1A) -----
246     #
247     # Pure function: computes the ``content_items.dedup_key`` value from metadata
248     # already extracted at ingestion time (no download ? honors ADR 0003). The
249     # decision tree is ordered strongest-key first; each branch is guarded so
250     # partial metadata can never collapse to a degenerate key (e.g. ``tg:fp:0:0:0:0``).
251     # See ``compute_dedup_key_test`` in ``tests/test_db.py`` for the full matrix.
252
253
254     def compute_dedup_key(
255         *,
256         provider: str,
257         document_id: str | None = None,
258         source_id: int | str | None = None,
259         provider_message_id: str | None = None,
260         size_bytes: int | None = None,
261         duration_s: int | None = None,
262         height: int | None = None,
263         width: int | None = None,
264     ) -> str:
265         """Compute the provider-neutral ``dedup_key`` for a ``ContentItem``.

```

```

266
267 Implements the ADR 0011 Option 1A decision tree:
268
269 - **Telegram** (`provider == "telegram"`):
270     1. If `document_id` is set ? `tg:doc:{document_id}`.
271     2. Else if **all four** fingerprint fields (size, duration, height, width)
272        are set ? `tg:fp:{size}:{dur}:{h}:{w}`.
273     3. Else ? `tg:msg:{source_id}:{provider_message_id}` (Tier-0 fallback;
274        `source_id` is required because Telegram `message_id` is only
275        unique within a channel).
276 - **Other providers** (`provider != "telegram"`): fall back to the Tier-0
277    ``{provider}:msg:{source_id}:{provider_message_id}`` shape. Future
278    providers add their own branches (e.g. `wa:sha256:{blob_hash}`) here.
279
280 Raises `ValueError` if the fallback branch is reached without the fields
281 it requires (`source_id` + `provider_message_id`) ? a degenerate
282 `dedup_key` is worse than a loud failure at ingestion time.
283 """
284 p = provider.lower()
285 if p == "telegram":
286     if document_id is not None and document_id != "":
287         return f"tg:doc:{document_id}"
288     # All four fingerprint fields are required to avoid a degenerate
289     # `tg:fp:0:0:0:0` key when some metadata is missing.
290     if None not in (size_bytes, duration_s, height, width):
291         return f"tg:fp:{size_bytes}:{duration_s}:{height}:{width}"
292     # Tier-0 fallback.
293     if source_id is None or provider_message_id is None:
294         raise ValueError(
295             "Telegram dedup_key fallback requires source_id and provider_message_id"
296         )
297     return f"tg:msg:{source_id}:{provider_message_id}"
298 # Future-proof default for providers without a dedicated branch.
299 if source_id is None or provider_message_id is None:
300     raise ValueError(
301         f"dedup_key for provider={provider!r} requires source_id and"
302         f"provider_message_id"
303     )
304 return f"{p}:msg:{source_id}:{provider_message_id}"
305
306
307 async def _apply_migrations(conn: aiosqlite.Connection) -> None:
308     """Run every registered migration whose version > current `user_version`."""
309     async with conn.execute("PRAGMA user_version") as cur:
310         row = await cur.fetchone()
311         current = int(row[0]) if row else 0
312
313     for version, fn in _MIGRATIONS:
314         if version <= current:
315             continue
316         await fn(conn)
317         await conn.execute(f"PRAGMA user_version = {version}")
318
319
320 async def _column_exists(conn: aiosqlite.Connection, table: str, column: str) -> bool:
321     """Return True if `column` exists on `table` (async, PRAGMA table_info)."""
322     async with conn.execute(f"PRAGMA table_info({table})") as cur:
323         rows = await cur.fetchall()
324         return any(row[1] == column for row in rows)
325
326
327 @migration(1)
328 async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:

```

```

329     """Add ``document_id`` column + partial unique index (ADR 0006 content dedup).
330
331     Idempotent: safe on fresh DBs (column/index already present) and on v1 DBs
332     created before this migration existed.
333     """
334     if not await _column_exists(conn, "videos", "document_id"):
335         await conn.execute("ALTER TABLE videos ADD COLUMN document_id INTEGER")
336     # Create the partial unique index for both fresh and migrated DBs. Uses
337     # IF NOT EXISTS so it is safe on DBs where it was already created.
338     await conn.execute(
339         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
340         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
341     )
342
343
344 @migration(2)
345 async def _migration_002_identity_split(conn: aiosqlite.Connection) -> None:
346     """Split the legacy ``users`` table into ``accounts`` + ``telegram_sessions``.
347
348     ADR 0010. Integer-preserving: ``accounts.id`` copies ``users.id`` and the
349     data in ``channels`` / ``videos`` / ``filter_config`` is copied verbatim,
350     so their integer ``user_id`` values still resolve to the same identity (now
351     via ``accounts.id``). The legacy ``users`` table is renamed to
352     ``users_deprecated`` and kept for one release as a rollback safety net; a
353     follow-up migration drops it.
354
355     FK contract: ``channels`` / ``videos`` / ``filter_config`` are rebuilt with
356     ``REFERENCES accounts(id)`` so migrated DBs end up with the *same* FK shape
357     as fresh DBs. (A naive rename of ``users`` would leave SQLite rewriting
358     those FKs to point at ``users_deprecated``, diverging from fresh installs
359     and breaking inserts for new accounts ? see PR #33 review.)
360
361     Idempotent: a no-op once the legacy ``users`` table has been renamed.
362     """
363     # If the legacy users table is gone, this migration has already run
364     # (or the DB is fresh and only has accounts). Nothing to do.
365     if not await _table_exists(conn, "users"):
366         return
367
368     # Build accounts from the legacy users table. accounts.id := users.id.
369     # (The empty accounts table created by _SCHEMA on a legacy DB is dropped
370     # and recreated here so the INSERT can populate it cleanly.)
371     await conn.execute("DROP TABLE IF EXISTS accounts")
372     await conn.execute(
373         """
374         CREATE TABLE accounts (
375             id            INTEGER PRIMARY KEY,
376             telegram_id    INTEGER UNIQUE,
377             provider       TEXT NOT NULL DEFAULT 'telegram',
378             display_name   TEXT,
379             created_at     TEXT NOT NULL
380         )
381         """
382     )
383     await conn.execute(
384         """
385         INSERT INTO accounts (id, telegram_id, provider, created_at)
386         SELECT id, id, 'telegram', created_at FROM users
387         """
388     )
389
390     # Build telegram_sessions, one row per legacy userbot user.
391     # (Same drop-and-recreate as accounts: _SCHEMA created an empty copy.)
392     await conn.execute("DROP TABLE IF EXISTS telegram_sessions")
393     await conn.execute(

```

```

394         """
395         CREATE TABLE telegram_sessions (
396             account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
397             phone          TEXT,
398             session_path  TEXT NOT NULL,
399             enabled        INTEGER NOT NULL DEFAULT 1,
400             created_at     TEXT NOT NULL,
401             last_login_at TEXT
402         )
403         """
404     )
405     # session_path is built from the *telegram_id* (= old users.id) to match
406     # the convention telethon_client.build_client used pre-migration. The
407     # sessions_dir prefix is prepended at read time (Settings.sessions_dir)
408     # so this migration does not need to know the runtime path.
409     await conn.execute(
410         """
411         INSERT INTO telegram_sessions (account_id, phone, session_path, enabled,
created_at)
412         SELECT id, phone, 'user_' || id || '.session', enabled, created_at
413         FROM users
414         """
415     )
416
417     # Rebuild channels / videos / filter_config so their FK constraints point
418     # at accounts(id) instead of the legacy users(id). SQLite's
419     # ALTER TABLE RENAME rewrites FK references in *other* tables to follow
420     # the rename ? so a naive rename of users ? users_deprecated would leave
421     # these tables referencing users_deprecated(id), diverging from fresh DBs
422     # (which reference accounts(id)) and breaking inserts for new accounts.
423     # The rebuild: rename the data-holding legacy table out of the way, create
424     # the new one with the correct FK, copy the data back, drop the renamed copy.
425     for table, new_ddl, copy_cols in (
426         (
427             "channels",
428             """
429             CREATE TABLE channels (
430                 id            INTEGER PRIMARY KEY AUTOINCREMENT,
431                 user_id       INTEGER NOT NULL REFERENCES accounts(id),
432                 channel_id    INTEGER NOT NULL,
433                 title         TEXT,
434                 seen_first_at TEXT NOT NULL,
435                 UNIQUE (user_id, channel_id)
436             )
437             """,
438             "id, user_id, channel_id, title, seen_first_at",
439         ),
440         (
441             "videos",
442             """
443             CREATE TABLE videos (
444                 id            INTEGER PRIMARY KEY AUTOINCREMENT,
445                 user_id       INTEGER NOT NULL REFERENCES accounts(id),
446                 channel_id    INTEGER NOT NULL,
447                 message_id    INTEGER NOT NULL,
448                 document_id   INTEGER,
449                 duration_s    INTEGER,
450                 width         INTEGER,
451                 height        INTEGER,
452                 size_bytes    INTEGER,
453                 mime_type     TEXT,
454                 matched        INTEGER NOT NULL DEFAULT 0,
455                 forwarded     INTEGER NOT NULL DEFAULT 0,
456                 seen_at       TEXT NOT NULL,
457                 UNIQUE (user_id, channel_id, message_id)

```

```

458         )
459         """
460         "id, user_id, channel_id, message_id, document_id, duration_s, width, "
461         "height, size_bytes, mime_type, matched, forwarded, seen_at",
462     ),
463     (
464         "filter_config",
465         """
466         CREATE TABLE filter_config (
467             user_id      INTEGER PRIMARY KEY REFERENCES accounts(id),
468             config_json TEXT NOT NULL
469         )
470         """
471         "user_id, config_json",
472     ),
473 ):
474     await conn.execute(f"ALTER TABLE {table} RENAME TO {table}__legacy_data")
475     await conn.execute(new_ddl)
476     await conn.execute(
477         f"INSERT INTO {table} ({copy_cols}) SELECT {copy_cols} FROM
{table}__legacy_data"
478     )
479     await conn.execute(f"DROP TABLE {table}__legacy_data")
480
481     # The partial unique index for document_id dedup (created by migration 001
482     # against the legacy videos table) was dropped along with the legacy copy
483     # above. Recreate it on the new videos table.
484     await conn.execute(
485         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
486         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
487     )
488
489     # Rename the legacy users table (do NOT drop ? kept as a rollback safety
490     # net for one release per ADR 0010). The data-holding channels/videos/
491     # filter_config tables were already rebuilt above with FKs to accounts(id),
492     # so renaming users no longer retargets any live FK.
493     await conn.execute("ALTER TABLE users RENAME TO users_deprecated")
494
495
496 @migration(3)
497 async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
498     """Land the provider-neutral domain tables (ADR 0011, Track B).
499
500     Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
501     ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
502     DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
503     them), then backfills the three populated ones from the legacy
504     ``channels`` / ``videos`` / ``filter_config`` tables and renames the
505     legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
506     pattern). Idempotent: a no-op once the legacy tables have been renamed.
507     """
508     # Idempotency gate: if the legacy videos table is gone, this migration has
509     # already run (or the DB is fresh and _SCHEMA's legacy tables were already
510     # dropped by _apply_schema post-003). Nothing to backfill.
511     if not await _table_exists(conn, "videos"):
512         return
513
514     # 1. Backfill sources from channels. ADR 0009 backfill: already-seen
515     # channels stay watched (watched=1). provider_source_id is the Telegram
516     # channel_id (cast to TEXT for the provider-neutral column).
517     await conn.execute(
518         """
519         INSERT OR IGNORE INTO sources
520             (account_id, provider, provider_source_id, title, kind, watched,
521             seen_first_at, last_seen_at)

```



```

522         SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',
523                1, seen_first_at, seen_first_at
524     FROM channels
525     """
526 )
527
528 # 2. Backfill content_items from videos, computing dedup_key in pure SQL.
529 #   The three-branch decision tree (Option 1A):
530 #       a) document_id IS NOT NULL           ? 'tg:doc:' || document_id
531 #       b) all four fingerprint fields set   ? 'tg:fp:' || size:dur:h:w
532 #       c) else (Tier-0 fallback)           ? 'tg:msg:' || source_id:msg_id
533 #   The ``CASE`` is ordered strongest-first and each branch is guarded so
534 #   partial metadata never collapses to a degenerate key. source_id is
535 #   resolved by joining the channel's provider_source_id back to sources.id
536 #   (Telegram message_id is per-channel, so source_id is required).
537 await conn.execute(
538     """
539     INSERT OR IGNORE INTO content_items
540         (account_id, source_id, provider, provider_message_id, content_type,
541          document_id, duration_s, width, height, size_bytes, mime_type,
542          dedup_key, matched, seen_at)
543     SELECT
544         v.user_id,
545         s.id,
546         'telegram',
547         CAST(v.message_id AS TEXT),
548         'video',
549         CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
550         v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
551         CASE
552             WHEN v.document_id IS NOT NULL
553             THEN 'tg:doc:' || v.document_id
554             WHEN v.size_bytes IS NOT NULL AND v.duration_s IS NOT NULL
555                  AND v.height IS NOT NULL AND v.width IS NOT NULL
556             THEN 'tg:fp:' || v.size_bytes || ':' || v.duration_s || ':'
557                  || v.height || ':' || v.width
558             ELSE 'tg:msg:' || s.id || ':' || v.message_id
559         END,
560         v.matched,
561         v.seen_at
562     FROM videos v
563     JOIN sources s
564         ON s.account_id = v.user_id
565        AND s.provider = 'telegram'
566        AND s.provider_source_id = CAST(v.channel_id AS TEXT)
567     """
568 )
569
570 # 3. Backfill policies from filter_config: one 'default' policy per account
571 #   (Option 2A). is_default=1 so the legacy load_filter_config shim reads
572 #   this row. UNIQUE(account_id, name) dedups idempotent re-runs.
573 await conn.execute(
574     """
575     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
576                                     created_at, updated_at)
577     SELECT user_id, 'default', config_json, 1, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP
578     FROM filter_config
579     """
580 )
581
582 # 4. Rename (don't drop) the legacy tables ? kept for one release as a
583 #   rollback safety net (ADR 0010 pattern).
584 await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
585 await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
586 await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")

```

```

587
588
589 async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
590     """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
591     async with conn.execute(
592         "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
593     ) as cur:
594         return await cur.fetchone() is not None
595
596
597 # --- helpers -----
598
599
600 def _now() -> datetime:
601     """Timezone-aware UTC 'now', used for policy timestamps."""
602     return datetime.now(UTC)
603
604
605 def _row_to_account(row: aiosqlite.Row) -> Account:
606     return Account(
607         id=row["id"],
608         telegram_id=row["telegram_id"],
609         provider=row["provider"],
610         display_name=row["display_name"],
611         created_at=row["created_at"], # stored as ISO string; pydantic parses
612     )
613
614
615 def _row_to_telegram_session(row: aiosqlite.Row) -> TelegramSession:
616     return TelegramSession(
617         account_id=row["account_id"],
618         phone=row["phone"],
619         session_path=row["session_path"],
620         enabled=bool(row["enabled"]),
621         created_at=row["created_at"],
622         last_login_at=row["last_login_at"],
623     )
624
625
626 def _account_session_to_user(account: Account, session: TelegramSession | None) -> User:
627     """Project an (account, optional session) pair onto the legacy ``User`` view.
628
629     Used by the deprecated ``upsert_user``/``get_user``/``list_enabled_users``
630     shims so pre-ADR-0010 callers and tests keep working. A bot-only account
631     (no session) maps to a ``User`` with ``enabled=False`` so the legacy
632     "list enabled users" semantics exclude it ? matching the structural rule
633     that bot-only users are not runnable Telethon clients.
634     """
635     if session is None:
636         return User(
637             id=account.telegram_id if account.telegram_id is not None else account.id,
638             phone=None,
639             enabled=False,
640             created_at=account.created_at,
641         )
642     return User(
643         id=account.telegram_id if account.telegram_id is not None else account.id,
644         phone=session.phone,
645         enabled=session.enabled,
646         created_at=session.created_at,
647     )
648
649
650 def _row_to_channel(row: aiosqlite.Row) -> Channel:
651     return Channel(

```

```

652         id=row["id"],
653         user_id=row["user_id"],
654         channel_id=row["channel_id"],
655         title=row["title"],
656         seen_first_at=row["seen_first_at"],
657     )
658
659
660 def _row_to_video(row: aiosqlite.Row) -> VideoRecord:
661     return VideoRecord(
662         id=row["id"],
663         user_id=row["user_id"],
664         channel_id=row["channel_id"],
665         message_id=row["message_id"],
666         document_id=row["document_id"],
667         duration_s=row["duration_s"],
668         width=row["width"],
669         height=row["height"],
670         size_bytes=row["size_bytes"],
671         mime_type=row["mime_type"],
672         matched=bool(row["matched"]),
673         forwarded=bool(row["forwarded"]),
674         seen_at=row["seen_at"],
675     )
676
677
678 def _row_to_source(row: aiosqlite.Row) -> Source:
679     return Source(
680         id=row["id"],
681         account_id=row["account_id"],
682         provider=row["provider"],
683         provider_source_id=row["provider_source_id"],
684         title=row["title"],
685         kind=row["kind"],
686         watched=bool(row["watched"]),
687         seen_first_at=row["seen_first_at"],
688         last_seen_at=row["last_seen_at"],
689     )
690
691
692 def _row_to_content_item(row: aiosqlite.Row) -> ContentItem:
693     return ContentItem(
694         id=row["id"],
695         account_id=row["account_id"],
696         source_id=row["source_id"],
697         provider=row["provider"],
698         provider_message_id=row["provider_message_id"],
699         content_type=row["content_type"],
700         text=row["text"],
701         link_url=row["link_url"],
702         media_ref=row["media_ref"],
703         document_id=row["document_id"],
704         duration_s=row["duration_s"],
705         width=row["width"],
706         height=row["height"],
707         size_bytes=row["size_bytes"],
708         mime_type=row["mime_type"],
709         dedup_key=row["dedup_key"],
710         matched=bool(row["matched"]),
711         seen_at=row["seen_at"],
712     )
713
714
715 def _row_to_policy(row: aiosqlite.Row) -> Policy:
716     return Policy(

```

```

717         id=row["id"],
718         account_id=row["account_id"],
719         name=row["name"],
720         config_json=row["config_json"],
721         is_default=bool(row["is_default"]),
722         created_at=row["created_at"],
723         updated_at=row["updated_at"],
724     )
725
726
727 # --- account + session repos (ADR 0010) -----
728
729
730 async def upsert_account(conn: aiosqlite.Connection, account: Account) -> Account:
731     """Insert or update an account row; returns the persisted Account."""
732     await conn.execute(
733         """INSERT INTO accounts (id, telegram_id, provider, display_name, created_at)
734         VALUES (?, ?, ?, ?, ?)
735         ON CONFLICT(id) DO UPDATE SET
736             telegram_id=excluded.telegram_id,
737             provider=excluded.provider,
738             display_name=excluded.display_name""",
739         (
740             account.id,
741             account.telegram_id,
742             account.provider,
743             account.display_name,
744             account.created_at.isoformat(),
745         ),
746     )
747     await conn.commit()
748     return await get_account(conn, account.id) # type: ignore[return-value]
749
750
751 async def get_account(conn: aiosqlite.Connection, account_id: int) -> Account | None:
752     conn.row_factory = aiosqlite.Row
753     async with conn.execute("SELECT * FROM accounts WHERE id = ?", (account_id,)) as cur:
754         row = await cur.fetchone()
755         return _row_to_account(row) if row else None
756
757
758 async def get_account_by_telegram_id(
759     conn: aiosqlite.Connection, telegram_id: int
760 ) -> Account | None:
761     """Resolve a Bot API ``from.id`` (or userbot account id) to an Account."""
762     conn.row_factory = aiosqlite.Row
763     async with conn.execute("SELECT * FROM accounts WHERE telegram_id = ?",
764 (telegram_id,)) as cur:
765         row = await cur.fetchone()
766         return _row_to_account(row) if row else None
767
768
769 async def upsert_telegram_session(
770     conn: aiosqlite.Connection, session: TelegramSession
771 ) -> TelegramSession:
772     """Insert or update a telegram_sessions row; returns the persisted session."""
773     await conn.execute(
774         """INSERT INTO telegram_sessions
775         (account_id, phone, session_path, enabled, created_at, last_login_at)
776         VALUES (?, ?, ?, ?, ?, ?)
777         ON CONFLICT(account_id) DO UPDATE SET
778             phone=excluded.phone,
779             session_path=excluded.session_path,
780             enabled=excluded.enabled,

```

```

780         last_login_at=excluded.last_login_at"",
781     (
782         session.account_id,
783         session.phone,
784         session.session_path,
785         int(session.enabled),
786         session.created_at.isoformat(),
787         session.last_login_at.isoformat() if session.last_login_at else None,
788     ),
789 )
790 await conn.commit()
791 conn.row_factory = aiosqlite.Row
792 async with conn.execute(
793     "SELECT * FROM telegram_sessions WHERE account_id = ?", (session.account_id,)
794 ) as cur:
795     row = await cur.fetchone()
796     # The row must exist ? we just upserted it by PK (account_id). Assert the
797     # invariant explicitly so mypy can narrow Row | None ? Row without a
798     # type-ignore comment masking a real future regression.
799     assert row is not None
800     return _row_to_telegram_session(row)
801
802
803 async def get_telegram_session(
804     conn: aiosqlite.Connection, account_id: int
805 ) -> TelegramSession | None:
806     conn.row_factory = aiosqlite.Row
807     async with conn.execute(
808         "SELECT * FROM telegram_sessions WHERE account_id = ?", (account_id,)
809     ) as cur:
810         row = await cur.fetchone()
811         return _row_to_telegram_session(row) if row else None
812
813
814 async def list_runnable_sessions(
815     conn: aiosqlite.Connection,
816 ) -> list[tuple[Account, TelegramSession]]:
817     """Return (account, session) pairs for sessions the manager should start.
818
819     This is the ADR 0010 structural rule: ``manager.run`` iterates this list,
820     not ``accounts``. A bot-only user has no ``telegram_sessions`` row and is
821     therefore inherently invisible to the Telethon supervisor ? the "don't
822     start a client for bot-only users" invariant is enforced by query shape.
823     """
824     conn.row_factory = aiosqlite.Row
825     async with conn.execute(
826         """SELECT a.*, s.phone AS s_phone, s.session_path, s.enabled AS s_enabled,
827             s.created_at AS s_created_at, s.last_login_at
828             FROM accounts a
829             JOIN telegram_sessions s ON s.account_id = a.id
830             WHERE s.enabled = 1"""
831     ) as cur:
832         rows = await cur.fetchall()
833     return [
834         (
835             _row_to_account(r),
836             TelegramSession(
837                 account_id=r["id"],
838                 phone=r["s_phone"],
839                 session_path=r["session_path"],
840                 enabled=bool(r["s_enabled"]),
841                 created_at=r["s_created_at"],
842                 last_login_at=r["last_login_at"],
843             ),
844         )

```

```

845         for r in rows
846     ]
847
848
849 # --- legacy User shims (backwards-compatible view over accounts + sessions) --
850
851
852 async def upsert_user(conn: aiosqlite.Connection, user: User) -> User:
853     """Deprecated shim: upsert the (account, session) pair implied by a ``User``.
854
855     Maps the legacy v1 ``User`` onto the ADR 0010 schema: ``User.id`` becomes
856     both ``accounts.id`` and ``accounts.telegram_id`` (legacy users are
857     Telegram userbot accounts), and a ``telegram_sessions`` row is created so
858     the manager can run them. ``User.enabled`` maps to ``session.enabled``.
859     """
860     account = Account(id=user.id, telegram_id=user.id, created_at=user.created_at)
861     await upsert_account(conn, account)
862     await upsert_telegram_session(
863         conn,
864         TelegramSession(
865             account_id=user.id,
866             phone=user.phone,
867             session_path=f"user_{user.id}.session",
868             enabled=user.enabled,
869             created_at=user.created_at,
870         ),
871     )
872     return await get_user(conn, user.id) # type: ignore[return-value]
873
874
875 async def get_user(conn: aiosqlite.Connection, user_id: int) -> User | None:
876     """Deprecated shim: project the (account, session) pair back to a ``User``."""
877     account = await get_account(conn, user_id)
878     if account is None:
879         return None
880     session = await get_telegram_session(conn, user_id)
881     return _account_session_to_user(account, session)
882
883
884 async def list_enabled_users(conn: aiosqlite.Connection) -> list[User]:
885     """Deprecated shim: list users that have an enabled MTPROTO session.
886
887     Equivalent to projecting :func:`list_runnable_sessions` onto the legacy
888     ``User`` shape. Bot-only accounts (no session) are excluded ? matching the
889     pre-ADR-0010 "list enabled users" semantics where every row was runnable.
890     """
891     pairs = await list_runnable_sessions(conn)
892     return [_account_session_to_user(a, s) for a, s in pairs]
893
894
895 # --- Source repos (ADR 0011) -----
896
897
898 async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
899     """Insert a source row, or update title/watched on conflict.
900
901     The conflict key is ``(account_id, provider, provider_source_id)`` per the
902     ADR 0011 schema. Returns the persisted row.
903     """
904     await conn.execute(
905         """INSERT INTO sources
906             (account_id, provider, provider_source_id, title, kind, watched,
907              seen_first_at, last_seen_at)
908             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
909             ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET

```

```

910         title=excluded.title,
911         watched=excluded.watched,
912         last_seen_at=excluded.last_seen_at"",
913     (
914         source.account_id,
915         source.provider,
916         source.provider_source_id,
917         source.title,
918         source.kind,
919         int(source.watched),
920         source.seen_first_at.isoformat(),
921         (source.last_seen_at.isoformat() if source.last_seen_at else None),
922     ),
923 )
924 await conn.commit()
925 conn.row_factory = aiosqlite.Row
926 async with conn.execute(
927     """SELECT * FROM sources
928         WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
929     (source.account_id, source.provider, source.provider_source_id),
930 ) as cur:
931     row = await cur.fetchone()
932     assert row is not None # just upserted by the conflict key
933     return _row_to_source(row)
934
935
936 async def get_source_by_provider_id(
937     conn: aiosqlite.Connection, account_id: int, provider: str, provider_source_id: str
938 ) -> Source | None:
939     """Resolve a source by its provider-native id."""
940     conn.row_factory = aiosqlite.Row
941     async with conn.execute(
942         """SELECT * FROM sources
943             WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
944         (account_id, provider, provider_source_id),
945     ) as cur:
946         row = await cur.fetchone()
947         return _row_to_source(row) if row else None
948
949
950 # --- ContentItem repos (ADR 0011) -----
951
952
953 async def insert_content_item(conn: aiosqlite.Connection, item: ContentItem) ->
ContentItem:
954     """Insert a content item. Raises ``IntegrityError`` on a dedup collision.
955
956     The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two
957     unique constraints guard against duplicates:
958     - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
959     - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
960     """
961     cur = await conn.execute(
962         """INSERT INTO content_items
963             (account_id, source_id, provider, provider_message_id, content_type,
964             text, link_url, media_ref, document_id, duration_s, width, height,
965             size_bytes, mime_type, dedup_key, matched, seen_at)
966             VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
967         (
968             item.account_id,
969             item.source_id,
970             item.provider,
971             item.provider_message_id,
972             item.content_type,
973             item.text,

```

```

974         item.link_url,
975         item.media_ref,
976         item.document_id,
977         item.duration_s,
978         item.width,
979         item.height,
980         item.size_bytes,
981         item.mime_type,
982         item.dedup_key,
983         int(item.matched),
984         item.seen_at.isoformat(),
985     ),
986 )
987 await conn.commit()
988 return item.model_copy(update={"id": cur.lastrowid})
989
990
991 async def get_content_item_by_message(
992     conn: aiomysql.Connection,
993     account_id: int,
994     source_id: int,
995     provider_message_id: str,
996 ) -> ContentItem | None:
997     """Tier-0 lookup by the (account, source, provider-message-id) natural key."""
998     conn.row_factory = aiomysql.Row
999     async with conn.execute(
1000         """SELECT * FROM content_items
1001            WHERE account_id = ? AND source_id = ? AND provider_message_id = ?""",
1002         (account_id, source_id, provider_message_id),
1003     ) as cur:
1004         row = await cur.fetchone()
1005     return _row_to_content_item(row) if row else None
1006
1007
1008 async def get_content_item_by_dedup_key(
1009     conn: aiomysql.Connection, account_id: int, dedup_key: str
1010 ) -> ContentItem | None:
1011     """Provider-neutral dedup lookup by ``dedup_key`` (Option 1A).
1012
1013     Replaces the Tier-1 (``document_id``) and Tier-2 (fingerprint) lookups with
1014     a single query: the ``dedup_key`` already encodes whichever tier was
1015     applicable at ingestion time.
1016
1017     """
1018     conn.row_factory = aiomysql.Row
1019     async with conn.execute(
1020         "SELECT * FROM content_items WHERE account_id = ? AND dedup_key = ?",
1021         (account_id, dedup_key),
1022     ) as cur:
1023         row = await cur.fetchone()
1024     return _row_to_content_item(row) if row else None
1025
1026 # --- Policy repos (ADR 0011, Option 2A) -----
1027
1028
1029 async def upsert_policy(conn: aiomysql.Connection, policy: Policy) -> Policy:
1030     """Insert or update a policy row by ``(account_id, name)``.
1031
1032     Exactly one policy per account should carry ``is_default=True``; callers
1033     are responsible for flipping the previous default before flipping a new
1034     one (enforced by a partial unique index landed by future work, not in the
1035     schema ? SQLite ``CHECK`` can't express "at most one per group").
1036
1037     """
1038     await conn.execute(
1039         """INSERT INTO policies (account_id, name, config_json, is_default,

```



```

1039             created_at, updated_at)
1040     VALUES (?, ?, ?, ?, ?, ?)
1041     ON CONFLICT(account_id, name) DO UPDATE SET
1042         config_json=excluded.config_json,
1043         is_default=excluded.is_default,
1044         updated_at=excluded.updated_at""",
1045     (
1046         policy.account_id,
1047         policy.name,
1048         policy.config_json,
1049         int(policy.is_default),
1050         policy.created_at.isoformat(),
1051         policy.updated_at.isoformat(),
1052     ),
1053 )
1054 await conn.commit()
1055 conn.row_factory = aiosqlite.Row
1056 async with conn.execute(
1057     "SELECT * FROM policies WHERE account_id = ? AND name = ?",
1058     (policy.account_id, policy.name),
1059 ) as cur:
1060     row = await cur.fetchone()
1061     assert row is not None
1062     return _row_to_policy(row)
1063
1064
1065 async def get_default_policy(conn: aiosqlite.Connection, account_id: int) -> Policy |
None:
1066     """Return the account's default policy (`is_default=1`)."""
1067     conn.row_factory = aiosqlite.Row
1068     async with conn.execute(
1069         "SELECT * FROM policies WHERE account_id = ? AND is_default = 1",
1070         (account_id,),
1071     ) as cur:
1072         row = await cur.fetchone()
1073         return _row_to_policy(row) if row else None
1074
1075
1076 # --- legacy Channel shim (backwards-compatible view over sources) -----
1077
1078
1079 async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1080     """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.
1081
1082     ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
1083     reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
1084     New code should call :func:`upsert_source` directly.
1085     """
1086     src = await upsert_source(
1087         conn,
1088         Source(
1089             account_id=channel.user_id,
1090             provider="telegram",
1091             provider_source_id=str(channel.channel_id),
1092             title=channel.title,
1093             kind="channel",
1094             watched=True, # back-compat: pre-ADR-0011 channels were all watched
1095             seen_first_at=channel.seen_first_at,
1096         ),
1097     )
1098     return _source_to_channel(src)
1099
1100
1101 async def get_channel(conn: aiosqlite.Connection, user_id: int, channel_id: int) ->
Channel | None:

```

```

1102     """Deprecated shim: project the :class:`Source` row back to a legacy Channel."""
1103     src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1104     return _source_to_channel(src) if src else None
1105
1106
1107 def _source_to_channel(src: Source) -> Channel:
1108     """Project a provider-neutral :class:`Source` onto the legacy :class:`Channel`."""
1109     return Channel(
1110         id=src.id,
1111         user_id=src.account_id,
1112         channel_id=int(src.provider_source_id),
1113         title=src.title,
1114         seen_first_at=src.seen_first_at,
1115     )
1116
1117
1118 # --- legacy VideoRecord shims (backwards-compatible view over content_items) -
1119
1120
1121 async def get_video(
1122     conn: aiosqlite.Connection, user_id: int, channel_id: int, message_id: int
1123 ) -> VideoRecord | None:
1124     """Deprecated shim: Tier-0 lookup against ``content_items`` (ADR 0011).
1125
1126     Resolves the legacy ``(user_id, channel_id, message_id)`` key to a
1127     :class:`Source` + :class:`ContentItem` and projects the latter onto a
1128     :class:`VideoRecord`. New code should call :func:`get_content_item_by_message`.
1129     """
1130     src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1131     if src is None:
1132         return None
1133     item = await get_content_item_by_message(conn, user_id, src.id, str(message_id))
1134     return _content_item_to_video(item, src, user_id, channel_id) if item else None
1135
1136
1137 async def get_video_by_document_id(
1138     conn: aiosqlite.Connection, user_id: int, document_id: int
1139 ) -> VideoRecord | None:
1140     """Deprecated shim: Tier-1 lookup via ``compute_dedup_key`` + ``content_items``."""
1141     dedup_key = compute_dedup_key(provider="telegram", document_id=str(document_id))
1142     item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1143     if item is None:
1144         return None
1145     src = await get_source_by_id(conn, item.source_id)
1146     if src is None:
1147         return None
1148     return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1149
1150
1151 async def get_video_by_content_fingerprint(
1152     conn: aiosqlite.Connection,
1153     user_id: int,
1154     size_bytes: int,
1155     duration_s: int,
1156     height: int,
1157     width: int,
1158 ) -> VideoRecord | None:
1159     """Deprecated shim: Tier-2 lookup via ``compute_dedup_key`` + ``content_items``."""
1160     dedup_key = compute_dedup_key(
1161         provider="telegram",
1162         size_bytes=size_bytes,
1163         duration_s=duration_s,
1164         height=height,
1165         width=width,
1166     )

```

```

1167     item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1168     if item is None:
1169         return None
1170     src = await get_source_by_id(conn, item.source_id)
1171     if src is None:
1172         return None
1173     return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1174
1175
1176 async def get_source_by_id(conn: aiomysqlite.Connection, source_id: int) -> Source | None:
1177     """Resolve a source by its surrogate PK (used by the legacy video shims)."""
1178     conn.row_factory = aiomysqlite.Row
1179     async with conn.execute("SELECT * FROM sources WHERE id = ?", (source_id,)) as cur:
1180         row = await cur.fetchone()
1181     return _row_to_source(row) if row else None
1182
1183
1184 async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
1185     """Deprecated shim: insert into ``content_items`` (ADR 0011).
1186
1187     Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
1188     if absent), computes the provider-neutral ``dedup_key`` via
1189     :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
1190     ``VideoRecord`` shape is reconstructed from the inserted row.
1191     """
1192     src = await upsert_source(
1193         conn,
1194         Source(
1195             account_id=video.user_id,
1196             provider="telegram",
1197             provider_source_id=str(video.channel_id),
1198             kind="channel",
1199             watched=True,
1200         ),
1201     )
1202     assert src.id is not None # upsert_source populates the surrogate PK
1203     item = ContentItem(
1204         account_id=video.user_id,
1205         source_id=src.id,
1206         provider="telegram",
1207         provider_message_id=str(video.message_id),
1208         content_type="video",
1209         document_id=str(video.document_id) if video.document_id is not None else None,
1210         duration_s=video.duration_s,
1211         width=video.width,
1212         height=video.height,
1213         size_bytes=video.size_bytes,
1214         mime_type=video.mime_type,
1215         dedup_key=compute_dedup_key(
1216             provider="telegram",
1217             document_id=(str(video.document_id) if video.document_id is not None else
1218             None),
1219             source_id=src.id,
1220             provider_message_id=str(video.message_id),
1221             size_bytes=video.size_bytes,
1222             duration_s=video.duration_s,
1223             height=video.height,
1224             width=video.width,
1225         ),
1226         matched=video.matched,
1227         seen_at=video.seen_at,
1228     )
1229     item = await insert_content_item(conn, item)
1230     return _content_item_to_video(item, src, video.user_id, video.channel_id)

```

```

1231
1232 async def mark_forwarded(conn: aioredis.Connection, video_id: int) -> None:
1233     """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).
1234
1235     Per-destination delivery state moves to ``delivery_attempts`` (Track C,
1236     #28). For Phase 1 (before Track C lands) there is no equivalent column on
1237     ``content_items`` ? the ``matched`` flag alone is sufficient because v1
1238     delivery still targets the single ``delivery_target`` and retried delivery
1239     is harmless (forwards are idempotent). This shim is retained so existing
1240     call sites in ``delivery.py`` keep working without a flag day.
1241     """
1242     # content_items has no ``forwarded`` column post-ADR-0011; nothing to write.
1243     # The row's ``matched`` flag (set at insert time) is the surviving signal.
1244     return None
1245
1246
1247 def _content_item_to_video(
1248     item: ContentItem, src: Source, user_id: int, channel_id: int
1249 ) -> VideoRecord:
1250     """Project a :class:`ContentItem` onto the legacy :class:`VideoRecord`.
1251
1252     ``forwarded`` is always ``False`` post-ADR-0011 (the column is dropped;
1253     per-destination delivery state moves to Track C). Existing callers read it
1254     only to decide whether to skip re-delivery; since forwards are idempotent
1255     the ``False`` default is safe for Phase 1.
1256     """
1257     return VideoRecord(
1258         id=item.id,
1259         user_id=user_id,
1260         channel_id=channel_id,
1261         message_id=int(item.provider_message_id),
1262         document_id=int(item.document_id) if item.document_id is not None else None,
1263         duration_s=item.duration_s,
1264         width=item.width,
1265         height=item.height,
1266         size_bytes=item.size_bytes,
1267         mime_type=item.mime_type,
1268         matched=item.matched,
1269         forwarded=False,
1270         seen_at=item.seen_at,
1271     )
1272
1273
1274 # --- filter config shims (backwards-compatible view over policies) -----
1275
1276
1277 async def load_filter_config(conn: aioredis.Connection, user_id: int) -> FilterConfig:
1278     """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1279
1280     ADR 0011 (Option 2A): ``filter_config`` is renamed to
1281     ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1282     instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1283     """
1284     policy = await get_default_policy(conn, user_id)
1285     if policy is None:
1286         return FilterConfig()
1287     return FilterConfig.model_validate_json(policy.config_json)
1288
1289
1290 async def save_filter_config(conn: aioredis.Connection, user_id: int, cfg:
FilterConfig) -> None:
1291     """Deprecated shim: upsert the account's default :class:`Policy`."""
1292     existing = await get_default_policy(conn, user_id)
1293     await upsert_policy(
1294         conn,

```

```

1295         Policy(
1296             id=existing.id if existing is not None else None,
1297             account_id=user_id,
1298             name="default",
1299             config_json=cfg.model_dump_json(),
1300             is_default=True,
1301             created_at=(existing.created_at if existing is not None else _now()),
1302             updated_at=_now(),
1303         ),
1304     )
1305
1306
1307 # --- bulk helpers (used by the manager on startup) -----
1308
1309
1310 async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1311     conn: aiosqlite.Connection, user_id: int, limit: int = 100
1312 ) -> list[VideoRecord]:
1313     """Deprecated shim: list the account's content items projected onto VideoRecord."""
1314     conn.row_factory = aiosqlite.Row
1315     async with conn.execute(
1316         """SELECT ci.*, s.provider_source_id AS src_pid
1317            FROM content_items ci
1318            JOIN sources s ON s.id = ci.source_id
1319            WHERE ci.account_id = ?
1320            ORDER BY ci.id DESC LIMIT ?""",
1321         (user_id, limit),
1322     ) as cur:
1323         rows = await cur.fetchall()
1324     return [
1325         VideoRecord(
1326             id=r["id"],
1327             user_id=user_id,
1328             channel_id=int(r["src_pid"]),
1329             message_id=int(r["provider_message_id"]),
1330             document_id=(int(r["document_id"]) if r["document_id"] is not None else
1331 None),
1332             duration_s=r["duration_s"],
1333             width=r["width"],
1334             height=r["height"],
1335             size_bytes=r["size_bytes"],
1336             mime_type=r["mime_type"],
1337             matched=bool(r["matched"]),
1338             forwarded=False,
1339             seen_at=r["seen_at"],
1340         )
1341         for r in rows
1342     ]
1343
1344 __all__ = [
1345     "init_db",
1346     "compute_dedup_key",
1347     # Account + session repos (ADR 0010)
1348     "upsert_account",
1349     "get_account",
1350     "get_account_by_telegram_id",
1351     "upsert_telegram_session",
1352     "get_telegram_session",
1353     "list_runnable_sessions",
1354     # Provider-neutral domain repos (ADR 0011)
1355     "upsert_source",
1356     "get_source_by_provider_id",
1357     "get_source_by_id",
1358     "insert_content_item",

```

```

1359     "get_content_item_by_message",
1360     "get_content_item_by_dedup_key",
1361     "upsert_policy",
1362     "get_default_policy",
1363     # Legacy User shims (backwards-compatible)
1364     "upsert_user",
1365     "get_user",
1366     "list_enabled_users",
1367     # Legacy Channel / Video / config shims (ADR 0011)
1368     "upsert_channel",
1369     "get_channel",
1370     "get_video",
1371     "get_video_by_document_id",
1372     "get_video_by_content_fingerprint",
1373     "insert_video",
1374     "mark_forwarded",
1375     "load_filter_config",
1376     "save_filter_config",
1377     "list_videos_for_user",
1378 ]
1379
1380
1381 def __dir__() -> Iterable[str]:
1382     return __all__
1383

```

5. assistant (2026-07-06T06:03:02.808Z)

I have the ADR and db.py. Now let me read the models and tests, plus check the README and main branch for the dedup tier claims.

6. user (2026-07-06T06:03:03.953Z)

```

1     """Pydantic data models for users, channels, videos, and filter config.
2
3     These models are the in-memory representation used by the repository layer
4     (`db.py`) and the filter engine (`filters/`). They are deliberately
5     decoupled from Telethon's own types so the filter engine stays pure and
6     unit-testable without a live Telegram connection.
7     """
8
9     from __future__ import annotations
10
11     from datetime import UTC, datetime
12     from enum import Enum
13
14     from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator
15
16
17     def _utcnow() -> datetime:
18         """Timezone-aware UTC 'now', used as a default factory."""
19         return datetime.now(UTC)
20
21
22     class DeliveryMode(str, Enum):
23         """How a matched video is delivered to the user.
24
25         - ``FORWARD``: forward the video itself to Saved Messages (re-uploads).
26         - ``LINK``: send a text message with a t.me link + metadata summary.
27         - ``BOTH``: forward the video AND send the link message.
28         """
29
30         FORWARD = "forward"
31         LINK = "link"
32         BOTH = "both"

```

```

33
34
35 class Account(BaseModel):
36     """A product identity ? the owner of config, channels, routes, and content.
37
38     Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
39     is the bridge used by the Bot API front-door (``from.id``) and by the
40     legacy userbot path; it is nullable so future providers that don't have a
41     Telegram id (e.g. WhatsApp) can still own an ``Account``.
42     """
43
44     model_config = ConfigDict(frozen=True)
45
46     id: int = Field(description="Stable surrogate PK; the owner key for all downstream
tables")
47     telegram_id: int | None = Field(
48         default=None,
49         description="Telegram user id (bot from.id OR userbot account id). UNIQUE when
set.",
50     )
51     provider: str = Field(default="telegram", description="Provider namespace; future:
'whatsapp'")
52     display_name: str | None = None
53     created_at: datetime = Field(default_factory=_utcnow)
54
55
56 class TelegramSession(BaseModel):
57     """An optional MTPROTO (Telethon) session linked to an :class:`Account`.
58
59     A bot-only user has an ``Account`` row but no ``TelegramSession`` row;
60     that is what makes them structurally invisible to ``manager.run``
61     (ADR 0010). ``session_path`` is data-driven so the file-location
62     convention is explicit rather than derived from the PK at runtime.
63     """
64
65     model_config = ConfigDict(frozen=True)
66
67     account_id: int = Field(description="FK ? accounts.id; PK (one session per account
in v1)")
68     phone: str | None = Field(default=None, description="E.164 phone; used only for
bootstrap")
69     session_path: str = Field(description="File-backed Telethon session path
(user_<id>.session)")
70     enabled: bool = Field(
71         default=True,
72         description="If False, the manager skips this session (not the account)",
73     )
74     created_at: datetime = Field(default_factory=_utcnow)
75     last_login_at: datetime | None = None
76
77
78 class User(BaseModel):
79     """Deprecated: legacy v1 model conflating identity + MTPROTO session.
80
81     Retained as a backwards-compatible view over the new ``accounts`` +
82     ``telegram_sessions`` tables (ADR 0010). New code should use
83     :class:`Account` + :class:`TelegramSession` directly. The ``db`` layer
84     maps ``User`` ? (``Account``, ``TelegramSession``) so existing callers
85     and tests keep working without a flag day.
86     """
87
88     model_config = ConfigDict(frozen=True)
89
90     id: int = Field(description="Telegram user id (== accounts.telegram_id for legacy
rows)")

```

```

91     phone: str | None = Field(default=None, description="E.164 phone; used only for
bootstrap")
92     enabled: bool = Field(default=True, description="If False, the manager skips this
session")
93     created_at: datetime = Field(default_factory=_utcnow)
94
95
96     class Channel(BaseModel):
97         """A channel/group a user is a member of. Populated lazily as messages arrive.
98
99         Deprecated: a provider-neutral view over :class:`Source` (ADR 0011). New
100         code should use :class:`Source` directly; the ``db`` layer maps
101         ``Channel`` ? ``Source`` so existing callers and tests keep working.
102         """
103
104         model_config = ConfigDict(frozen=True)
105
106         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
107         user_id: int
108         channel_id: int = Field(description="Telegram channel/chat id")
109         title: str | None = None
110         seen_first_at: datetime = Field(default_factory=_utcnow)
111
112
113     # --- provider-neutral domain models (ADR 0011, Track B) -----
114     #
115     # These generalize the legacy v1 ``Channel`` / ``VideoRecord`` / ``FilterConfig``
116     # shapes so the schema no longer leaks Telegram-specific ids into every future
117     # provider's code path. ``provider`` columns + provider-prefixed ``dedup_key``
118     # make cross-provider collisions structurally impossible. See ADR 0011 for the
119     # full rationale and the two decisions (Option 1A: tiered ``dedup_key``;
120     # Option 2A: ``policies`` generalizes ``filter_config``).
121
122
123     class Source(BaseModel):
124         """A configured place we ingest content from (ADR 0011).
125
126         Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127         providers add their own source kinds). ``provider_source_id`` is the
128         provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129         """
130
131         model_config = ConfigDict(frozen=True)
132
133         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134         account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135         provider: str = Field(default="telegram", description="Provider namespace")
136         provider_source_id: str = Field(
137             description="Provider-native source id (Telegram channel_id, ?)"
138         )
139         title: str | None = None
140         kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141         watched: bool = Field(
142             default=False,
143             description="ADR 0009 selective scanning: ingest from this source iff True",
144         )
145         seen_first_at: datetime = Field(default_factory=_utcnow)
146         last_seen_at: datetime | None = None
147
148
149     class ContentItem(BaseModel):
150         """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152         Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153         (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay

```



```

154     NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155     ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156     wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157     """
158
159     model_config = ConfigDict(frozen=True)
160
161     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162     account_id: int = Field(description="FK ? accounts.id")
163     source_id: int = Field(description="FK ? sources.id")
164     provider: str = Field(default="telegram", description="Provider namespace")
165     provider_message_id: str = Field(
166         description="Provider-native message id (Telegram message_id, ?)"
167     )
168     content_type: str = Field(default="video", description="'video' | 'link' |
'article'")
169     text: str | None = Field(default=None, description="Caption / post text (nullable in
Phase 1)")
170     link_url: str | None = Field(default=None, description="For link/article posts
(Phase 1: NULL)")
171     media_ref: str | None = Field(default=None, description="Provider media reference")
172     document_id: str | None = Field(
173         default=None,
174         description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
175     )
176     duration_s: int | None = None
177     width: int | None = None
178     height: int | None = None
179     size_bytes: int | None = None
180     mime_type: str | None = None
181     dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
per account")
182     matched: bool = False
183     seen_at: datetime = Field(default_factory=_utcnow)
184
185
186     class Policy(BaseModel):
187         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188
189         Generalizes the legacy per-account ``filter_config`` row so one account can
190         own multiple policies (enables ADR 0009's per-route filter overrides) and so
191         future enrichment/security stages grow into the same ``config_json`` column.
192         Exactly one policy per account carries ``is_default=True``; the legacy
193         ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
194         """
195
196         model_config = ConfigDict(frozen=True)
197
198         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
199         account_id: int = Field(description="FK ? accounts.id")
200         name: str = Field(description="Policy name; 'default' for the account default
policy")
201         config_json: str = Field(
202             description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
203         )
204         is_default: bool = Field(
205             default=False, description="Exactly one per account (UNIQUE partial index)"
206         )
207         created_at: datetime = Field(default_factory=_utcnow)
208         updated_at: datetime = Field(default_factory=_utcnow)
209
210
211     class FilterConfig(BaseModel):
212         """Per-user filter configuration.
213

```

```

214 v1 rules: duration bounds, minimum height (resolution), and max file size.
215 Stored in the DB as a JSON blob (`filter_config.config_json`) so adding a
216 new field later is backwards-compatible.
217 """
218
219 # Duration bounds in seconds. None means unbounded on that side.
220 min_duration_s: int | None = Field(default=5, ge=0)
221 max_duration_s: int | None = Field(default=7200, ge=0)
222
223 # Resolution: a video passes if its height >= target_height.
224 # None disables the resolution rule.
225 target_height: int | None = Field(default=1080, ge=1)
226
227 # Max file size in mebibytes. None disables the size rule.
228 max_size_mb: int | None = Field(default=2048, ge=1)
229
230 # Content-type exclusions. GIFs and round videos are excluded by default
231 # (their duration/dimensions are usually incidental to real video filters);
232 # silent videos are off by default since some legit clips have no audio.
233 # Existing users' stored configs are unaffected ? the DB JSON blob overrides
234 # these defaults, so this only changes the behaviour for fresh installs.
235 exclude_gif: bool = Field(
236     default=True,
237     description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
238 )
239 exclude_round: bool = Field(
240     default=True,
241     description="If True, exclude circular video messages (round_message=True)",
242 )
243 exclude_nosound: bool = Field(
244     default=False,
245     description="If True, exclude videos without an audio track (nosound=True)",
246 )
247
248 # How matched videos are delivered: forward the video, send a link, or both.
249 # Stored in the same JSON blob, so adding this field is backwards-compatible
250 # (existing rows without it default to FORWARD).
251 delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)
252
253 # Where matched videos / link messages are sent. Accepts:
254 # - "me" (default): Saved Messages ? exact v1.1 behaviour.
255 # - "@username": a public channel the account is a member of.
256 # - "-100..." or a bare int-as-string: a numeric Telegram chat id
257 #   (private channels without a username).
258 # The string is passed verbatim to Telethon's forward_messages / send_message.
259 # See ADR 0005 for rationale and access requirements.
260 delivery_target: str = Field(
261     default="me",
262     min_length=1,
263     description="Telethon entity to deliver to: 'me', '@username', or a chat id",
264 )
265
266 @field_validator("delivery_target", mode="before")
267 @classmethod
268 def _coerce_delivery_target(cls, v: object) -> str:
269     """Accept int chat ids (e.g. -1001234567890) and coerce to str.
270
271     Telethon accepts both str and int as an entity, but storing as str keeps
272     the JSON blob uniform and lets '@username' / 'me' / numeric ids share
273     one field type.
274     """
275     if v is None:
276         return "me"
277     if isinstance(v, int):
278         return str(v)

```

```

279         return str(v)
280
281     @model_validator(mode="after")
282     def _check_duration_bounds(self) -> FilterConfig:
283         """Ensure min_duration_s <= max_duration_s when both are set."""
284         if (
285             self.min_duration_s is not None
286             and self.max_duration_s is not None
287             and self.min_duration_s > self.max_duration_s
288         ):
289             raise ValueError(
290                 f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s ({self.max_duration_s})"
291             )
292         return self
293
294
295     class VideoRecord(BaseModel):
296         """A seen video message + its metadata. Drives dedup and future /search.
297
298         Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).
299         New code should use :class:`ContentItem`; the ``db`` layer maps
300         ``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.
301         """
302
303         model_config = ConfigDict(frozen=True)
304
305         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
306         user_id: int
307         channel_id: int = Field(description="Origin Telegram channel id")
308         message_id: int = Field(description="Telegram message id within the channel")
309         document_id: int | None = Field(
310             default=None,
311             description="Telegram document.id ? reused across forwards; Tier-1 content dedup",
312         )
313
314         # Metadata extracted from the Telethon message (may be None if absent).
315         duration_s: int | None = None
316         width: int | None = None
317         height: int | None = None
318         size_bytes: int | None = None
319         mime_type: str | None = None
320
321         matched: bool = False
322         forwarded: bool = False
323         seen_at: datetime = Field(default_factory=_utcnow)
324
325     @property
326     def dedup_key(self) -> tuple[int, int, int]:
327         """The natural key used for deduplication in the ``videos`` table."""
328         return (self.user_id, self.channel_id, self.message_id)
329
330     @property
331     def content_fingerprint(self) -> tuple[int, int, int, int] | None:
332         """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height, width).
333
334         Returns None if any field is missing (cannot fingerprint).
335         """
336         if None in (self.size_bytes, self.duration_s, self.height, self.width):
337             return None
338         return (self.size_bytes, self.duration_s, self.height, self.width) # type: ignore[return-value]
339

```

```

340
341     # Default filter config shared by the bootstrap onboarding step.
342     DEFAULT_FILTER_CONFIG = FilterConfig()
343

```

7. user (2026-07-06T06:03:04.386Z)

```

1     """Tests for the SQLite data layer (db.py) using an in-memory database."""
2
3     from __future__ import annotations
4
5     import sqlite3
6
7     import aiosqlite
8     import pytest
9
10    from tg_video_filter import db
11    from tg_video_filter.models import (
12        DEFAULT_FILTER_CONFIG,
13        Account,
14        Channel,
15        ContentItem,
16        FilterConfig,
17        Policy,
18        Source,
19        TelegramSession,
20        User,
21        VideoRecord,
22    )
23
24    USER_ID = 111
25    CHANNEL_ID = -1001234567890
26
27
28    @pytest.fixture
29    async def conn() -> aiosqlite.Connection:
30        """An in-memory SQLite connection with the schema applied."""
31        c = await aiosqlite.connect(":memory:")
32        await db._apply_schema(c)
33        yield c
34        await c.close()
35
36
37    async def _seed_user(conn: aiosqlite.Connection, uid: int = USER_ID) -> User:
38        return await db.upsert_user(conn, User(id=uid, phone="+15555550100"))
39
40
41    # --- schema / init -----
42
43
44    async def test_init_db_creates_tables(tmp_path: object) -> None:
45        db_path = str(tmp_path) + "/test.db" # type: ignore[operator]
46        await db.init_db(db_path)
47        # Re-open and confirm all expected tables exist. Post-ADR-0011 the
48        # provider-neutral tables are sources/content_items/policies (plus the
49        # empty Track-C tables destinations/routing_rules/delivery_attempts);
50        # the legacy channels/videos/filter_config are absent on fresh DBs
51        # (they only exist as *_deprecated on migrated legacy DBs).
52        async with (
53            aiosqlite.connect(db_path) as c,
54            c.execute("SELECT name FROM sqlite_master WHERE type='table' ORDER BY name") as
cur,
55        ):
56            names = [r[0] for r in await cur.fetchall()]
57            assert set(names) >= {

```

```

58         "accounts",
59         "telegram_sessions",
60         "sources",
61         "content_items",
62         "policies",
63         "destinations",
64         "routing_rules",
65         "delivery_attempts",
66     }
67
68
69 # --- user repos -----
70
71
72 async def test_upsert_and_get_user(conn: aiomysqlite.Connection) -> None:
73     created = await _seed_user(conn)
74     assert created.id == USER_ID
75     assert created.enabled is True
76
77     fetched = await db.get_user(conn, USER_ID)
78     assert fetched is not None
79     assert fetched.phone == "+15555550100"
80
81
82 async def test_get_user_returns_none_when_absent(conn: aiomysqlite.Connection) -> None:
83     assert await db.get_user(conn, 999999) is None
84
85
86 async def test_upsert_user_updates_existing(conn: aiomysqlite.Connection) -> None:
87     await _seed_user(conn)
88     # Disable the user via an upsert.
89     await db.upsert_user(conn, User(id=USER_ID, phone="+15555550200", enabled=False))
90     fetched = await db.get_user(conn, USER_ID)
91     assert fetched is not None
92     assert fetched.enabled is False
93     assert fetched.phone == "+15555550200"
94
95
96 async def test_list_enabled_users_filters_disabled(conn: aiomysqlite.Connection) -> None:
97     await db.upsert_user(conn, User(id=1, enabled=True))
98     await db.upsert_user(conn, User(id=2, enabled=False))
99     await db.upsert_user(conn, User(id=3, enabled=True))
100     enabled = await db.list_enabled_users(conn)
101     assert sorted(u.id for u in enabled) == [1, 3]
102
103
104 # --- account + session repos (ADR 0010) -----
105
106
107 async def _seed_account(conn: aiomysqlite.Connection, aid: int = USER_ID) -> Account:
108     return await db.upsert_account(conn, Account(id=aid, telegram_id=aid))
109
110
111 async def test_upsert_and_get_account(conn: aiomysqlite.Connection) -> None:
112     created = await db.upsert_account(
113         conn, Account(id=USER_ID, telegram_id=USER_ID, display_name="Alice")
114     )
115     assert created.id == USER_ID
116     assert created.telegram_id == USER_ID
117     assert created.provider == "telegram"
118
119     fetched = await db.get_account(conn, USER_ID)
120     assert fetched is not None
121     assert fetched.display_name == "Alice"
122

```

```

123
124 async def test_get_account_returns_none_when_absent(conn: aiosqlite.Connection) -> None:
125     assert await db.get_account(conn, 999999) is None
126
127
128 async def test_get_account_by_telegram_id_resolves_bot_from_id(
129     conn: aiosqlite.Connection,
130 ) -> None:
131     """The Bot API authorization bridge: from.id ? Account (ADR 0008/0010)."""
132     await db.upsert_account(conn, Account(id=USER_ID, telegram_id=USER_ID))
133     found = await db.get_account_by_telegram_id(conn, USER_ID)
134     assert found is not None
135     assert found.id == USER_ID
136     # Unknown telegram id resolves to None.
137     assert await db.get_account_by_telegram_id(conn, 888888) is None
138
139
140 async def test_upsert_and_get_telegram_session(conn: aiosqlite.Connection) -> None:
141     await _seed_account(conn)
142     session = await db.upsert_telegram_session(
143         conn,
144         TelegramSession(
145             account_id=USER_ID,
146             phone="+15555550100",
147             session_path="user_111.session",
148             enabled=True,
149         ),
150     )
151     assert session.account_id == USER_ID
152     assert session.session_path == "user_111.session"
153     assert session.enabled is True
154
155     fetched = await db.get_telegram_session(conn, USER_ID)
156     assert fetched is not None
157     assert fetched.phone == "+15555550100"
158
159
160 async def test_list_runnable_sessions_excludes_disabled_and_bot_only(
161     conn: aiosqlite.Connection,
162 ) -> None:
163     """ADR 0010 structural invariant: bot-only + disabled sessions are invisible."""
164     # Account 1: userbot, enabled ? runnable.
165     await db.upsert_account(conn, Account(id=1, telegram_id=1))
166     await db.upsert_telegram_session(
167         conn, TelegramSession(account_id=1, session_path="user_1.session", enabled=True)
168     )
169     # Account 2: userbot, disabled ? not runnable.
170     await db.upsert_account(conn, Account(id=2, telegram_id=2))
171     await db.upsert_telegram_session(
172         conn, TelegramSession(account_id=2, session_path="user_2.session",
173         enabled=False)
174     )
175     # Account 3: bot-only (no session row) ? not runnable.
176     await db.upsert_account(conn, Account(id=3, telegram_id=3))
177
178     pairs = await db.list_runnable_sessions(conn)
179     assert [acct.id for acct, _ in pairs] == [1]
180
181
182 async def test_bot_only_account_has_no_session_and_is_not_a_user(
183     conn: aiosqlite.Connection,
184 ) -> None:
185     """A bot-only account is a real Account but not a legacy 'enabled user'."""
186     await db.upsert_account(conn, Account(id=USER_ID, telegram_id=USER_ID))

```

```

187     # Account exists.
188     assert await db.get_account(conn, USER_ID) is not None
189     # No session row.
190     assert await db.get_telegram_session(conn, USER_ID) is None
191     # Legacy User shim returns a user with enabled=False (not runnable).
192     legacy = await db.get_user(conn, USER_ID)
193     assert legacy is not None
194     assert legacy.enabled is False
195     # And it is excluded from the enabled-users list.
196     assert await db.list_enabled_users(conn) == []
197
198
199     # --- channel repos -----
200
201
202     async def test_upsert_channel_inserts_and_is_idempotent(conn: aioredis.Connection) ->
None:
203         await _seed_user(conn)
204         ch = await db.upsert_channel(
205             conn, Channel(user_id=USER_ID, channel_id=CHANNEL_ID, title="First title")
206         )
207         assert ch.id is not None
208         assert ch.title == "First title"
209
210         # Second upsert with a new title updates the existing row (no duplicate).
211         ch2 = await db.upsert_channel(
212             conn, Channel(user_id=USER_ID, channel_id=CHANNEL_ID, title="New title")
213         )
214         assert ch2.id == ch.id
215         assert ch2.title == "New title"
216
217
218     # --- video repos -----
219
220
221     async def test_get_video_absent(conn: aioredis.Connection) -> None:
222         assert await db.get_video(conn, USER_ID, CHANNEL_ID, 1) is None
223
224
225     async def test_insert_and_get_video_round_trip(conn: aioredis.Connection) -> None:
226         await _seed_user(conn)
227         v = VideoRecord(
228             user_id=USER_ID,
229             channel_id=CHANNEL_ID,
230             message_id=42,
231             duration_s=120,
232             width=1920,
233             height=1080,
234             size_bytes=5_000_000,
235             mime_type="video/mp4",
236             matched=True,
237             forwarded=False,
238         )
239         inserted = await db.insert_video(conn, v)
240         assert inserted.id is not None
241
242         fetched = await db.get_video(conn, USER_ID, CHANNEL_ID, 42)
243         assert fetched is not None
244         assert fetched.duration_s == 120
245         assert fetched.height == 1080
246         assert fetched.size_bytes == 5_000_000
247         assert fetched.mime_type == "video/mp4"
248         assert fetched.matched is True
249         assert fetched.forwarded is False
250

```

```

251
252 async def test_insert_video_dedup_collision(conn: aiosqlite.Connection) -> None:
253     await _seed_user(conn)
254     v = VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=7)
255     await db.insert_video(conn, v)
256     # Same (user, channel, message) must violate the unique constraint.
257     with pytest.raises(sqlite3.IntegrityError):
258         await db.insert_video(conn, v)
259
260
261 async def test_mark_forwarded_is_noop_post_adr_0011(conn: aiosqlite.Connection) -> None:
262     """ADR 0011: mark_forwarded is a no-op shim (delivery state ? Track C).
263
264     Post-migration-003 there is no ``forwarded`` column on ``content_items``;
265     per-destination delivery state moves to ``delivery_attempts`` in Track C.
266     For Phase 1 the call must succeed without error (existing callers in
267     ``delivery.py`` keep working) and the surviving ``matched`` flag is
268     unaffected.
269     """
270     await _seed_user(conn)
271     inserted = await db.insert_video(
272         conn, VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=9,
matched=True)
273     )
274     assert inserted.id is not None
275     # No-op: must not raise.
276     await db.mark_forwarded(conn, inserted.id)
277     fetched = await db.get_video(conn, USER_ID, CHANNEL_ID, 9)
278     assert fetched is not None
279     # matched survives; forwarded is always False post-ADR-0011.
280     assert fetched.matched is True
281     assert fetched.forwarded is False
282
283
284 async def test_list_videos_for_user_orders_desc(conn: aiosqlite.Connection) -> None:
285     await _seed_user(conn)
286     for msg_id in (1, 2, 3):
287         await db.insert_video(
288             conn, VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=msg_id)
289         )
290     vids = await db.list_videos_for_user(conn, USER_ID)
291     # Newest insert first (DESC by id).
292     assert [v.message_id for v in vids] == [3, 2, 1]
293
294
295 async def test_video_record_dedup_key() -> None:
296     v = VideoRecord(user_id=1, channel_id=2, message_id=3)
297     assert v.dedup_key == (1, 2, 3)
298
299
300 async def test_video_record_content_fingerprint() -> None:
301     v = VideoRecord(
302         user_id=1,
303         channel_id=2,
304         message_id=3,
305         size_bytes=5_000_000,
306         duration_s=120,
307         height=1080,
308         width=1920,
309     )
310     assert v.content_fingerprint == (5_000_000, 120, 1080, 1920)
311
312
313 async def test_video_record_content_fingerprint_none_when_missing_field() -> None:
314     """Fingerprint is None if any of the four metadata fields is missing."""

```



```

315     v = VideoRecord(
316         user_id=1,
317         channel_id=2,
318         message_id=3,
319         size_bytes=5_000_000,
320         duration_s=120,
321         height=1080,
322         # width is None
323     )
324     assert v.content_fingerprint is None
325
326
327     # --- content dedup (Tiers 1 & 2) -----
328
329
330     async def test_get_video_by_document_id_hit(conn: aiomysqlite.Connection) -> None:
331         await _seed_user(conn)
332         await db.insert_video(
333             conn,
334             VideoRecord(
335                 user_id=USER_ID,
336                 channel_id=CHANNEL_ID,
337                 message_id=1,
338                 document_id=555000111,
339             ),
340         )
341         found = await db.get_video_by_document_id(conn, USER_ID, 555000111)
342         assert found is not None
343         assert found.document_id == 555000111
344         assert found.message_id == 1
345
346
347     async def test_get_video_by_document_id_miss(conn: aiomysqlite.Connection) -> None:
348         await _seed_user(conn)
349         assert await db.get_video_by_document_id(conn, USER_ID, 999999) is None
350
351
352     async def test_insert_video_document_id_collision(conn: aiomysqlite.Connection) -> None:
353         """Same document_id for the same user ? IntegrityError (Tier-1 dedup at DB
354         level)."""
355         await _seed_user(conn)
356         await db.insert_video(
357             conn,
358             VideoRecord(
359                 user_id=USER_ID,
360                 channel_id=CHANNEL_ID,
361                 message_id=10,
362                 document_id=123456789,
363             ),
364         )
365         # Same doc_id, different channel/message ? must collide on the index.
366         with pytest.raises(sqlite3.IntegrityError):
367             await db.insert_video(
368                 conn,
369                 VideoRecord(
370                     user_id=USER_ID,
371                     channel_id=-1009999999,
372                     message_id=99,
373                     document_id=123456789,
374                 ),
375             )
376
377     async def test_insert_video_null_document_id_no_collision(conn: aiomysqlite.Connection) ->
None:

```

```

378     """Multiple rows with document_id=NULL do NOT collide (partial unique index)."""
379     await _seed_user(conn)
380     await db.insert_video(
381         conn,
382         VideoRecord(
383             user_id=USER_ID,
384             channel_id=CHANNEL_ID,
385             message_id=20,
386             document_id=None,
387         ),
388     )
389     # Same user, no document_id, different message ? should succeed.
390     inserted = await db.insert_video(
391         conn,
392         VideoRecord(
393             user_id=USER_ID,
394             channel_id=CHANNEL_ID,
395             message_id=21,
396             document_id=None,
397         ),
398     )
399     assert inserted.id is not None
400
401
402     async def test_get_video_by_content_fingerprint_hit(conn: aiosqlite.Connection) -> None:
403         await _seed_user(conn)
404         await db.insert_video(
405             conn,
406             VideoRecord(
407                 user_id=USER_ID,
408                 channel_id=CHANNEL_ID,
409                 message_id=5,
410                 size_bytes=10_000_000,
411                 duration_s=180,
412                 height=720,
413                 width=1280,
414             ),
415         )
416         found = await db.get_video_by_content_fingerprint(
417             conn,
418             USER_ID,
419             10_000_000,
420             180,
421             720,
422             1280,
423         )
424         assert found is not None
425         assert found.message_id == 5
426
427
428     async def test_get_video_by_content_fingerprint_miss(conn: aiosqlite.Connection) ->
None:
429         await _seed_user(conn)
430         # No video inserted ? should be None.
431         assert (
432             await db.get_video_by_content_fingerprint(
433                 conn,
434                 USER_ID,
435                 1,
436                 1,
437                 1,
438                 1,
439             )
440             is None
441         )

```

```

442
443
444 # --- filter config repos -----
445
446
447 async def test_load_filter_config_defaults_when_absent(conn: aiomysqlite.Connection) ->
None:
448     await _seed_user(conn)
449     cfg = await db.load_filter_config(conn, USER_ID)
450     assert cfg == DEFAULT_FILTER_CONFIG
451
452
453 async def test_save_and_load_filter_config_round_trip(conn: aiomysqlite.Connection) ->
None:
454     await _seed_user(conn)
455     custom = FilterConfig(
456         min_duration_s=10, max_duration_s=3600, target_height=720, max_size_mb=1024
457     )
458     await db.save_filter_config(conn, USER_ID, custom)
459     loaded = await db.load_filter_config(conn, USER_ID)
460     assert loaded == custom
461
462
463 async def test_save_filter_config_overwrites(conn: aiomysqlite.Connection) -> None:
464     await _seed_user(conn)
465     await db.save_filter_config(conn, USER_ID, FilterConfig(max_duration_s=100))
466     await db.save_filter_config(conn, USER_ID, FilterConfig(max_duration_s=200))
467     loaded = await db.load_filter_config(conn, USER_ID)
468     assert loaded.max_duration_s == 200
469
470
471 async def test_filter_config_with_null_bounds_round_trips(conn: aiomysqlite.Connection) ->
None:
472     """None bounds (rule disabled) must survive the JSON round trip."""
473     await _seed_user(conn)
474     cfg = FilterConfig(
475         min_duration_s=None, max_duration_s=None, target_height=None, max_size_mb=None
476     )
477     await db.save_filter_config(conn, USER_ID, cfg)
478     loaded = await db.load_filter_config(conn, USER_ID)
479     assert loaded.min_duration_s is None
480     assert loaded.max_duration_s is None
481     assert loaded.target_height is None
482     assert loaded.max_size_mb is None
483
484
485 async def test_delivery_mode_round_trips(conn: aiomysqlite.Connection) -> None:
486     """The delivery_mode field survives the JSON round trip for all values."""
487     from tg_video_filter.models import DeliveryMode
488
489     await _seed_user(conn)
490     for mode in DeliveryMode:
491         await db.save_filter_config(conn, USER_ID, FilterConfig(delivery_mode=mode))
492         loaded = await db.load_filter_config(conn, USER_ID)
493         assert loaded.delivery_mode == mode
494
495
496 async def test_filter_config_rejects_inverted_duration_bounds() -> None:
497     """A config with min_duration > max_duration must raise ValidationError."""
498     from pydantic import ValidationError
499
500     from tg_video_filter.models import FilterConfig
501
502     with pytest.raises(ValidationError, match="min_duration_s.*must be <="):
503         FilterConfig(min_duration_s=7200, max_duration_s=5)

```

```

504
505
506 def test_filter_config_accepts_equal_min_max_duration() -> None:
507     """min_duration_s == max_duration_s is valid (exact-match filter)."""
508     from tg_video_filter.models import FilterConfig
509
510     cfg = FilterConfig(min_duration_s=60, max_duration_s=60)
511     assert cfg.min_duration_s == 60
512     assert cfg.max_duration_s == 60
513
514
515 async def test_old_config_without_delivery_mode_defaults_to_forward(
516     conn: aiosqlite.Connection,
517 ) -> None:
518     """A pre-v1.1 config JSON (no delivery_mode key) must default to FORWARD."""
519     await _seed_user(conn)
520     # Simulate an old row by inserting raw JSON without the delivery_mode field
521     # into the default policy (ADR 0011: filter_config is now policies).
522     old_json =
523     '{"min_duration_s":5,"max_duration_s":7200,"target_height":1080,"max_size_mb":2048}'
524     await conn.execute(
525         "INSERT OR REPLACE INTO policies (account_id, name, config_json, is_default, "
526         "created_at, updated_at) VALUES (?, 'default', ?, 1, CURRENT_TIMESTAMP, "
527         "CURRENT_TIMESTAMP)",
528         (USER_ID, old_json),
529     )
530     await conn.commit()
531     loaded = await db.load_filter_config(conn, USER_ID)
532     from tg_video_filter.models import DeliveryMode
533
534     assert loaded.delivery_mode == DeliveryMode.FORWARD
535
536
537 async def test_delivery_target_round_trips(conn: aiosqlite.Connection) -> None:
538     """The delivery_target field survives the JSON round trip for all forms."""
539     await _seed_user(conn)
540     for target in ("me", "@my-feed", "-1001234567890", "1234567890"):
541         await db.save_filter_config(conn, USER_ID, FilterConfig(delivery_target=target))
542         loaded = await db.load_filter_config(conn, USER_ID)
543         assert loaded.delivery_target == target
544
545
546 async def test_old_config_without_delivery_target_defaults_to_me(
547     conn: aiosqlite.Connection,
548 ) -> None:
549     """A pre-v1.2 config JSON (no delivery_target key) must default to 'me'."""
550     await _seed_user(conn)
551     old_json = (
552         '{"min_duration_s":5,"max_duration_s":7200,"target_height":1080,'
553         '"max_size_mb":2048,"delivery_mode":"forward"}'
554     )
555     await conn.execute(
556         "INSERT OR REPLACE INTO policies (account_id, name, config_json, is_default, "
557         "created_at, updated_at) VALUES (?, 'default', ?, 1, CURRENT_TIMESTAMP, "
558         "CURRENT_TIMESTAMP)",
559         (USER_ID, old_json),
560     )
561     await conn.commit()
562     loaded = await db.load_filter_config(conn, USER_ID)
563     assert loaded.delivery_target == "me"
564
565
566 def test_filter_config_coerces_int_delivery_target() -> None:
567     """An int chat id is coerced to str (so JSON stays uniform)."""
568     from tg_video_filter.models import FilterConfig

```

```

566
567     cfg = FilterConfig(delivery_target=-1001234567890) # type: ignore[arg-type]
568     assert cfg.delivery_target == "-1001234567890"
569
570
571 def test_filter_config_rejects_empty_delivery_target() -> None:
572     """An empty delivery_target is invalid (min_length=1)."""
573     from pydantic import ValidationError
574
575     from tg_video_filter.models import FilterConfig
576
577     with pytest.raises(ValidationError):
578         FilterConfig(delivery_target="")
579
580
581 async def test_pragmas_are_set_on_init(conn: aiosqlite.Connection) -> None:
582     """Verify WAL mode, foreign keys, and busy timeout are enabled."""
583     await db._apply_schema(conn)
584     async with conn.execute("PRAGMA journal_mode") as cur:
585         row = await cur.fetchone()
586         assert row is not None
587         # In-memory databases report "memory"; file-backed DBs report "wal".
588         assert row[0].lower() in ("wal", "memory")
589     async with conn.execute("PRAGMA foreign_keys") as cur:
590         assert (await cur.fetchone())[0] == 1
591     async with conn.execute("PRAGMA busy_timeout") as cur:
592         assert (await cur.fetchone())[0] == 5000
593
594
595 async def test_foreign_key_enforcement_rejects_orphan_video(
596     conn: aiosqlite.Connection,
597 ) -> None:
598     """Inserting a video for a non-existent user must fail with FK error."""
599     import sqlite3
600
601     with pytest.raises(sqlite3.IntegrityError, match="FOREIGN KEY"):
602         await db.insert_video(conn, VideoRecord(user_id=99999, channel_id=1,
603 message_id=1))
604
605 # --- migrations -----
606
607
608 async def test_fresh_db_has_document_id_column(conn: aiosqlite.Connection) -> None:
609     """A fresh schema (via _apply_schema) must include document_id on content_items.
610
611     ADR 0011: the partial unique index ``idx_videos_user_document`` is
612     superseded by ``UNIQUE(account_id, dedup_key)`` on ``content_items``; the
613     ``document_id`` column survives as a TEXT field for TG-specific Tier-1
614     lookups. This test now checks the new location.
615     """
616     assert await db._column_exists(conn, "content_items", "document_id")
617     # The provider-neutral dedup uniqueness lives on content_items, not videos.
618     async with conn.execute("PRAGMA index_list('content_items')") as cur:
619         idx_names = {row[1] for row in await cur.fetchall()}
620     # SQLite names the UNIQUE(...) constraints auto-generated indexes; assert
621     # at least one unique index exists (the dedup_key uniqueness).
622     assert any("dedup_key" in idx or "sqlite_autoindex" in idx for idx in idx_names),
623 idx_names
624
625
626 async def test_fresh_db_user_version_is_latest(conn: aiosqlite.Connection) -> None:
627     """init_db/_apply_schema must bump user_version to the highest migration."""
628     await db._apply_schema(conn)
629     async with conn.execute("PRAGMA user_version") as cur:

```

```

629         version = (await cur.fetchone())[0]
630     assert version == len(db._MIGRATIONS)
631     assert version >= 1
632
633
634     async def test_migration_adds_document_id_to_legacy_db(tmp_path: object) -> None:
635         """A v1 DB (no document_id, user_version=0) must be migrated on open."""
636         db_path = str(tmp_path) + "/legacy.db" # type: ignore[operator]
637
638         # Build a legacy v1 schema WITH the real FK definitions (REFERENCES users),
639         # no document_id column, user_version = 0. Using the real FK constraints
640         # is what lets this test catch the PR #33 regression where SQLite rewrites
641         # FK references to follow a rename (users ? users_deprecated) instead of
642         # retargeting them to accounts.
643         legacy_schema = """
644         CREATE TABLE users (
645             id INTEGER PRIMARY KEY, phone TEXT,
646             enabled INTEGER NOT NULL DEFAULT 1, created_at TEXT NOT NULL
647         );
648         CREATE TABLE channels (
649             id INTEGER PRIMARY KEY AUTOINCREMENT,
650             user_id INTEGER NOT NULL REFERENCES users(id),
651             channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
652             UNIQUE (user_id, channel_id)
653         );
654         CREATE TABLE videos (
655             id INTEGER PRIMARY KEY AUTOINCREMENT,
656             user_id INTEGER NOT NULL REFERENCES users(id),
657             channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL,
658             duration_s INTEGER, width INTEGER, height INTEGER, size_bytes INTEGER,
659             mime_type TEXT, matched INTEGER NOT NULL DEFAULT 0,
660             forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL,
661             UNIQUE (user_id, channel_id, message_id)
662         );
663         CREATE TABLE filter_config (
664             user_id INTEGER PRIMARY KEY REFERENCES users(id), config_json TEXT NOT NULL
665         );
666         """
667         async with aiosqlite.connect(db_path) as conn:
668             await conn.executescript(legacy_schema)
669             # Seed rows so we can confirm data survives the migration.
670             await conn.execute(
671                 "INSERT INTO users (id, phone, enabled, created_at) VALUES (111, '+1', 1,
672                 '2024-01-01T00:00:00Z')"
673             )
674             await conn.execute(
675                 "INSERT INTO channels (user_id, channel_id, title, seen_first_at) "
676                 "VALUES (111, -1001, 'pre-migration', '2024-01-01T00:00:00Z')"
677             )
678             await conn.execute(
679                 "INSERT INTO videos (user_id, channel_id, message_id, duration_s, width,
680                 height, "
681                 "size_bytes, mime_type, matched, forwarded, seen_at) "
682                 "VALUES (111, -1001, 1, 60, 1920, 1080, 1000000, 'video/mp4', 1, 1,
683                 '2024-01-01T00:00:00Z')"
684             )
685             await conn.execute("INSERT INTO filter_config (user_id, config_json) VALUES
686             (111, '{}')")
687             await conn.commit()
688             # user_version is 0 on a freshly created DB.
689
690         # Now open via init_db ? migrations 001, 002, and 003 must run.
691         await db.init_db(db_path)
692
693     async with aiosqlite.connect(db_path) as conn:

```

```

690         await conn.execute("PRAGMA foreign_keys=ON")
691         # Column added (migration 001) ? on the legacy videos table, now renamed.
692         assert await db._column_exists(conn, "videos_deprecated", "document_id")
693         # Identity split (migration 002): users renamed, accounts present.
694         assert await db._table_exists(conn, "accounts")
695         assert await db._table_exists(conn, "telegram_sessions")
696         assert await db._table_exists(conn, "users_deprecated")
697         assert not await db._table_exists(conn, "users")
698         # accounts.id copied from users.id (integer-preserving).
699         async with conn.execute("SELECT id, telegram_id, provider FROM accounts") as
cur:
700             rows = await cur.fetchall()
701             assert rows == [(111, 111, "telegram")]
702             # telegram_sessions row derived from the legacy user row.
703             async with conn.execute(
704                 "SELECT account_id, phone, session_path, enabled FROM telegram_sessions"
705             ) as cur:
706                 srows = await cur.fetchall()
707                 assert srows == [(111, "+1", "user_111.session", 1)]
708
709             # Domain schema (migration 003): legacy tables renamed to *_deprecated,
710             # provider-neutral tables backfilled from them.
711             assert await db._table_exists(conn, "sources")
712             assert await db._table_exists(conn, "content_items")
713             assert await db._table_exists(conn, "policies")
714             assert await db._table_exists(conn, "channels_deprecated")
715             assert await db._table_exists(conn, "videos_deprecated")
716             assert await db._table_exists(conn, "filter_config_deprecated")
717             assert not await db._table_exists(conn, "channels")
718             assert not await db._table_exists(conn, "videos")
719             assert not await db._table_exists(conn, "filter_config")
720
721             # FK contract (PR #33 review): the rebuilt legacy tables (now
722             # *_deprecated) reference accounts(id), NOT users_deprecated(id).
723             for tbl in ("channels_deprecated", "videos_deprecated",
"filter_config_deprecated"):
724                 async with conn.execute(f"PRAGMA foreign_key_list({tbl})") as cur:
725                     fk_rows = await cur.fetchall()
726                     referenced_tables = {r[2] for r in fk_rows}
727                     assert "accounts" in referenced_tables, f"{tbl} FK does not reference
accounts"
728                     assert (
729                         "users_deprecated" not in referenced_tables
730                     ), f"{tbl} FK still references users_deprecated (rename-rewrite bug)"
731
732             # Legacy data preserved in the deprecated tables.
733             async with conn.execute(
734                 "SELECT message_id, document_id, user_id FROM videos_deprecated"
735             ) as cur:
736                 vrows = await cur.fetchall()
737                 assert vrows == [(1, None, 111)]
738             async with conn.execute(
739                 "SELECT user_id, channel_id, title FROM channels_deprecated"
740             ) as cur:
741                 crows = await cur.fetchall()
742                 assert crows == [(111, -1001, "pre-migration")]
743
744             # Domain backfill (migration 003): sources + content_items populated.
745             async with conn.execute(
746                 "SELECT account_id, provider, provider_source_id, title, watched " "FROM
sources"
747             ) as cur:
748                 src_rows = await cur.fetchall()
749                 assert src_rows == [(111, "telegram", "-1001", "pre-migration", 1)]
750                 async with conn.execute(

```

```

751         "SELECT account_id, source_id, provider_message_id, document_id, "
752         "duration_s, width, height, size_bytes, dedup_key FROM content_items"
753     ) as cur:
754         ci_rows = await cur.fetchall()
755         # dedup_key falls to the Tier-0 branch (no document_id, but the
756         # fingerprint fields are all present) ? actually the fingerprint branch
757         # wins here since size/duration/h/w are all set.
758         assert len(ci_rows) == 1
759         ci = ci_rows[0]
760         assert ci[0] == 111 # account_id
761         assert ci[2] == "1" # provider_message_id (cast to TEXT)
762         assert ci[3] is None # document_id
763         assert ci[4:8] == (60, 1920, 1080, 1000000) # dur, w, h, size
764         assert ci[8] == "tg:fp:1000000:60:1080:1920" # fingerprint dedup_key
765
766         # Policy backfill (migration 003): one default policy per account.
767         async with conn.execute(
768             "SELECT account_id, name, config_json, is_default FROM policies"
769         ) as cur:
770             pol_rows = await cur.fetchall()
771             assert pol_rows == [(111, "default", "{}", 1)]
772
773         # user_version bumped to the highest registered migration.
774         async with conn.execute("PRAGMA user_version") as cur:
775             assert (await cur.fetchone())[0] == len(db._MIGRATIONS)
776
777
778     async def test_migration_002_retargeted_fks_allow_inserts_for_new_accounts(
779         tmp_path: object,
780     ) -> None:
781         """Regression guard for PR #33: post-migration, a channel for a brand-new
782         account (not present in users_deprecated) must insert cleanly under
783         PRAGMA foreign_keys=ON. Catches the SQLite rename-rewrites-FK bug.
784         """
785         db_path = str(tmp_path) + "/legacy_fk.db" # type: ignore[operator]
786         legacy_schema = ""
787         CREATE TABLE users (
788             id INTEGER PRIMARY KEY, phone TEXT,
789             enabled INTEGER NOT NULL DEFAULT 1, created_at TEXT NOT NULL
790         );
791         CREATE TABLE channels (
792             id INTEGER PRIMARY KEY AUTOINCREMENT,
793             user_id INTEGER NOT NULL REFERENCES users(id),
794             channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
795             UNIQUE (user_id, channel_id)
796         );
797         INSERT INTO users (id, phone, enabled, created_at) VALUES (111, '+1', 1,
798             '2024-01-01T00:00:00Z');
799
800         async with aiosqlite.connect(db_path) as conn:
801             await conn.executescript(legacy_schema)
802             await conn.commit()
803
804         await db.init_db(db_path)
805
806         async with aiosqlite.connect(db_path) as conn:
807             await conn.execute("PRAGMA foreign_keys=ON")
808             # Create a NEW account that exists only in accounts, not users_deprecated.
809             await conn.execute(
810                 "INSERT INTO accounts (id, telegram_id, provider, created_at) "
811                 "VALUES (222, 222, 'telegram', '2024-01-01T00:00:00Z')"
812             )
813             # This insert must succeed: channels_deprecated.user_id now references
814             # accounts(id), and account 222 exists. Under the bug it would raise FK
815             # constraint failed. (Post-migration-003 the table is channels_deprecated.)

```



```

815         await conn.execute(
816             "INSERT INTO channels_deprecated (user_id, channel_id, title, seen_first_at)
817             "VALUES (222, -2002, 'new-account', '2024-01-01T00:00:00Z')")
818         )
819         await conn.commit()
820
821
822     async def test_migration_002_is_idempotent_on_fresh_db(tmp_path: object) -> None:
823         """Migration 002 is a no-op on a fresh DB (accounts already exists)."""
824         db_path = str(tmp_path) + "/fresh.db" # type: ignore[operator]
825         await db.init_db(db_path)
826         async with aiosqlite.connect(db_path) as conn:
827             # Fresh DB has accounts, no users, no users_deprecated.
828             assert await db._table_exists(conn, "accounts")
829             assert not await db._table_exists(conn, "users")
830             assert not await db._table_exists(conn, "users_deprecated")
831
832
833     async def test_migration_is_idempotent(tmp_path: object) -> None:
834         """Running init_db twice must not error (idempotent migration + schema)."""
835         db_path = str(tmp_path) + "/idempotent.db" # type: ignore[operator]
836         await db.init_db(db_path)
837         # Second run re-applies schema + migrations; must succeed and stay current.
838         await db.init_db(db_path)
839         async with aiosqlite.connect(db_path) as conn:
840             async with conn.execute("PRAGMA user_version") as cur:
841                 assert (await cur.fetchone())[0] == len(db._MIGRATIONS)
842             assert await db._column_exists(conn, "content_items", "document_id")
843             assert await db._table_exists(conn, "content_items")
844             assert await db._table_exists(conn, "sources")
845             assert await db._table_exists(conn, "policies")
846
847
848     async def test_document_id_unique_index_enforces_dedup(conn: aiosqlite.Connection) ->
None:
849         """The partial unique index must reject a second row with the same document_id."""
850         import sqlite3
851
852         await _seed_user(conn)
853         await db.insert_video(
854             conn,
855             VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=1,
document_id=999),
856         )
857         # Same document_id, different (channel, message) ? must violate the index.
858         with pytest.raises(sqlite3.IntegrityError, match="UNIQUE"):
859             await db.insert_video(
860                 conn,
861                 VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=2,
document_id=999),
862             )
863         # But NULL document_id rows must coexist (partial index).
864         await db.insert_video(
865             conn,
866             VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=3,
document_id=None),
867         )
868         await db.insert_video(
869             conn,
870             VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=4,
document_id=None),
871         )
872
873

```

```

874 # --- compute_dedup_key (ADR 0011, Option 1A) -----
875
876
877 def test_compute_dedup_key_telegram_document_id_wins() -> None:
878     """Tier-1 (document_id) is the strongest key; it takes precedence."""
879     key = db.compute_dedup_key(
880         provider="telegram",
881         document_id="12345",
882         source_id=1,
883         provider_message_id="99",
884         size_bytes=1000,
885         duration_s=60,
886         height=1080,
887         width=1920,
888     )
889     assert key == "tg:doc:12345"
890
891
892 def test_compute_dedup_key_telegram_fingerprint_when_no_document_id() -> None:
893     """Tier-2 (fingerprint) applies when document_id is absent AND all four
894     metadata fields are present (the guard against degenerate tg:fp:0:0:0:0)."""
895     key = db.compute_dedup_key(
896         provider="telegram",
897         size_bytes=1000,
898         duration_s=60,
899         height=1080,
900         width=1920,
901     )
902     assert key == "tg:fp:1000:60:1080:1920"
903
904
905 def test_compute_dedup_key_telegram_tier0_when_partial_metadata() -> None:
906     """Tier-0 (message id) applies when fingerprint is incomplete ? prevents
907     the partial-metadata collision (e.g. missing width ? must NOT use fp branch)."""
908     key = db.compute_dedup_key(
909         provider="telegram",
910         source_id=7,
911         provider_message_id="42",
912         size_bytes=1000,
913         duration_s=60,
914         height=1080,
915         width=None, # missing ? fingerprint branch skipped
916     )
917     assert key == "tg:msg:7:42"
918
919
920 def test_compute_dedup_key_telegram_tier0_when_all_metadata_missing() -> None:
921     """Tier-0 applies when no media metadata at all."""
922     key = db.compute_dedup_key(provider="telegram", source_id=7,
923     provider_message_id="42")
924     assert key == "tg:msg:7:42"
925
926
927 def test_compute_dedup_key_telegram_empty_document_id_falls_through() -> None:
928     """An empty-string document_id is treated as absent (falls to next branch)."""
929     key = db.compute_dedup_key(
930         provider="telegram",
931         document_id="",
932         source_id=7,
933         provider_message_id="42",
934     )
935     assert key == "tg:msg:7:42"
936
937
938 def test_compute_dedup_key_telegram_tier0_requires_source_and_message() -> None:

```

```

938     """The Tier-0 fallback needs source_id + provider_message_id (Telegram
939     message_id is per-channel); raising is better than a degenerate key."""
940     with pytest.raises(ValueError, match="source_id and provider_message_id"):
941         db.compute_dedup_key(provider="telegram") # no metadata at all
942
943
944 def test_compute_dedup_key_other_provider_uses_provider_prefix() -> None:
945     """Future providers get a provider-prefixed Tier-0 key (no tg: branch)."""
946     key = db.compute_dedup_key(
947         provider="whatsapp", source_id="wa-group-1", provider_message_id="wa-msg-9"
948     )
949     assert key == "whatsapp:msg:wa-group-1:wa-msg-9"
950
951
952 def test_compute_dedup_key_other_provider_requires_source_and_message() -> None:
953     """Non-telegram providers also need source_id + provider_message_id."""
954     with pytest.raises(ValueError, match="provider='whatsapp'"):
955         db.compute_dedup_key(provider="whatsapp")
956
957
958 def test_compute_dedup_key_provider_is_case_insensitive() -> None:
959     """Provider prefix is lowercased so Telegram/telegram/TELEGRAM collide safely."""
960     assert (
961         db.compute_dedup_key(provider="TELEGRAM", document_id="5")
962         == db.compute_dedup_key(provider="telegram", document_id="5")
963         == "tg:doc:5"
964     )
965
966
967 # --- Source / ContentItem / Policy repos (ADR 0011) -----
968
969
970 async def test_upsert_source_round_trip(conn: aiomysqlite.Connection) -> None:
971     """Insert then read back a source via the provider-neutral repo."""
972     await _seed_user(conn)
973     src = await db.upsert_source(
974         conn,
975         Source(
976             account_id=USER_ID,
977             provider="telegram",
978             provider_source_id="-100555",
979             title="Test Channel",
980             kind="channel",
981             watched=True,
982         ),
983     )
984     assert src.id is not None
985     fetched = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-100555")
986     assert fetched is not None
987     assert fetched.title == "Test Channel"
988     assert fetched.watched is True
989     assert fetched.kind == "channel"
990
991
992 async def test_upsert_source_updates_on_conflict(conn: aiomysqlite.Connection) -> None:
993     """Re-upserting the same (account, provider, source_id) updates title/watched."""
994     await _seed_user(conn)
995     await db.upsert_source(
996         conn,
997         Source(account_id=USER_ID, provider="telegram", provider_source_id="-1",
998 watched=True),
999     )
1000     updated = await db.upsert_source(
1001         conn,
1002         Source(

```

```

1002         account_id=USER_ID,
1003         provider="telegram",
1004         provider_source_id="-1",
1005         title="Renamed",
1006         watched=False,
1007     ),
1008 )
1009 assert updated.title == "Renamed"
1010 assert updated.watched is False
1011 # Still one row.
1012 fetched = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1")
1013 assert fetched is not None
1014 assert fetched.watched is False
1015
1016
1017 async def test_insert_content_item_round_trip(conn: aiosqlite.Connection) -> None:
1018     """Insert a content item and read it back via both natural keys."""
1019     await _seed_user(conn)
1020     src = await db.upsert_source(
1021         conn, Source(account_id=USER_ID, provider="telegram", provider_source_id="-1")
1022     )
1023     assert src.id is not None
1024     item = await db.insert_content_item(
1025         conn,
1026         ContentItem(
1027             account_id=USER_ID,
1028             source_id=src.id,
1029             provider="telegram",
1030             provider_message_id="42",
1031             document_id="999",
1032             duration_s=60,
1033             width=1920,
1034             height=1080,
1035             size_bytes=1000,
1036             mime_type="video/mp4",
1037             dedup_key=db.compute_dedup_key(provider="telegram", document_id="999"),
1038         ),
1039     )
1040     assert item.id is not None
1041     by_msg = await db.get_content_item_by_message(conn, USER_ID, src.id, "42")
1042     assert by_msg is not None
1043     assert by_msg.document_id == "999"
1044     by_key = await db.get_content_item_by_dedup_key(conn, USER_ID, "tg:doc:999")
1045     assert by_key is not None
1046     assert by_key.id == item.id
1047
1048
1049 async def test_content_item_dedup_key_uniqueness(conn: aiosqlite.Connection) -> None:
1050     """UNIQUE(account_id, dedup_key) rejects a second item with the same key."""
1051     import sqlite3
1052
1053     await _seed_user(conn)
1054     src = await db.upsert_source(
1055         conn, Source(account_id=USER_ID, provider="telegram", provider_source_id="-1")
1056     )
1057     assert src.id is not None
1058     await db.insert_content_item(
1059         conn,
1060         ContentItem(
1061             account_id=USER_ID,
1062             source_id=src.id,
1063             provider="telegram",
1064             provider_message_id="1",
1065             document_id="777",
1066             dedup_key="tg:doc:777",

```

```

1067         ),
1068     )
1069     with pytest.raises(sqlite3.IntegrityError, match="UNIQUE"):
1070         await db.insert_content_item(
1071             conn,
1072             ContentItem(
1073                 account_id=USER_ID,
1074                 source_id=src.id,
1075                 provider="telegram",
1076                 provider_message_id="2", # different message, same dedup key
1077                 document_id="777",
1078                 dedup_key="tg:doc:777",
1079             ),
1080         )
1081
1082
1083     async def test_policy_upsert_and_default_lookup(conn: aiosqlite.Connection) -> None:
1084         """Upsert a default policy and read it back via get_default_policy."""
1085         await _seed_user(conn)
1086         policy = await db.upsert_policy(
1087             conn,
1088             Policy(
1089                 account_id=USER_ID,
1090                 name="default",
1091                 config_json='{"min_duration_s":5}',
1092                 is_default=True,
1093             ),
1094         )
1095         assert policy.id is not None
1096         fetched = await db.get_default_policy(conn, USER_ID)
1097         assert fetched is not None
1098         assert fetched.config_json == '{"min_duration_s":5}'
1099         assert fetched.is_default is True
1100
1101
1102     async def test_filter_config_shim_uses_default_policy(conn: aiosqlite.Connection) ->
None:
1103         """save_filter_config ? load_filter_config round-trips through policies."""
1104         await _seed_user(conn)
1105         custom = FilterConfig(min_duration_s=30, max_duration_s=600, target_height=720)
1106         await db.save_filter_config(conn, USER_ID, custom)
1107         loaded = await db.load_filter_config(conn, USER_ID)
1108         assert loaded.min_duration_s == 30
1109         assert loaded.target_height == 720
1110         # The default policy row exists.
1111         policy = await db.get_default_policy(conn, USER_ID)
1112         assert policy is not None
1113         assert policy.is_default is True
1114
1115
1116     # --- migration 003: legacy data backfill (ADR 0011) -----
1117
1118
1119     async def test_migration_003_backfills_all_dedup_key_branches(tmp_path: object) -> None:
1120         """The content_items backfill must exercise all three dedup_key branches
1121         (document_id / fingerprint / Tier-0 message) and never collapse two rows
1122         to the same key. Seeds rows hitting each branch on a legacy DB.
1123         """
1124         db_path = str(tmp_path) + "/legacy_dedup.db" # type: ignore[operator]
1125         legacy_schema = """
1126         CREATE TABLE users (
1127             id INTEGER PRIMARY KEY, phone TEXT,
1128             enabled INTEGER NOT NULL DEFAULT 1, created_at TEXT NOT NULL
1129         );
1130         CREATE TABLE channels (

```

```

1131         id INTEGER PRIMARY KEY AUTOINCREMENT,
1132         user_id INTEGER NOT NULL REFERENCES users(id),
1133         channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
1134         UNIQUE (user_id, channel_id)
1135     );
1136     CREATE TABLE videos (
1137         id INTEGER PRIMARY KEY AUTOINCREMENT,
1138         user_id INTEGER NOT NULL REFERENCES users(id),
1139         channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL,
1140         document_id INTEGER, duration_s INTEGER, width INTEGER, height INTEGER,
1141         size_bytes INTEGER, mime_type TEXT, matched INTEGER NOT NULL DEFAULT 0,
1142         forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL,
1143         UNIQUE (user_id, channel_id, message_id)
1144     );
1145     CREATE TABLE filter_config (
1146         user_id INTEGER PRIMARY KEY REFERENCES users(id), config_json TEXT NOT NULL
1147     );
1148     """
1149     async with aiosqlite.connect(db_path) as conn:
1150         await conn.executescript(legacy_schema)
1151         await conn.execute(
1152             "INSERT INTO users (id, phone, enabled, created_at) "
1153             "VALUES (111, '+1', 1, '2024-01-01T00:00:00Z')"
1154         )
1155         # Two channels so message_id-only keys don't collide.
1156         await conn.executemany(
1157             "INSERT INTO channels (user_id, channel_id, title, seen_first_at) "
1158             "VALUES (111, ?, ?, '2024-01-01T00:00:00Z')",
1159             [(-1001, "Channel A"), (-1002, "Channel B")],
1160         )
1161         await conn.executemany(
1162             "INSERT INTO videos (user_id, channel_id, message_id, document_id, "
1163             "duration_s, width, height, size_bytes, mime_type, matched, forwarded,
1164             seen_at) "
1165             "VALUES (111, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, '2024-01-01T00:00:00Z')",
1166             [
1167                 # Branch 1: document_id set ? tg:doc:
1168                 (-1001, 1, 5000, 60, 1920, 1080, 1000, "video/mp4", 1, 0),
1169                 # Branch 2: no document_id, all four fingerprint fields ? tg:fp:
1170                 (-1001, 2, None, 60, 1920, 1080, 2000, "video/mp4", 0, 0),
1171                 # Branch 3: no document_id, partial metadata (no width) ? tg:msg:
1172                 (-1002, 3, None, 60, None, 1080, 3000, "video/mp4", 0, 0),
1173                 # Branch 3: no document_id, no metadata at all ? tg:msg:
1174                 (-1002, 4, None, None, None, None, None, None, 0, 0),
1175             ],
1176         )
1177         await conn.execute(
1178             "INSERT INTO filter_config (user_id, config_json) VALUES (111, "
1179             '{"min_duration_s":5})'
1180         )
1181         await conn.commit()
1182
1183     async with aiosqlite.connect(db_path) as conn:
1184         # All four videos migrated, each with a distinct dedup_key. The
1185         # exact source_id in the tg:msg: keys is the surrogate PK of the
1186         # channel -1002's sources row (inserted second ? id=2); the critical
1187         # invariant is that all four keys are distinct (no partial-metadata
1188         # collision collapsed two rows to the same key).
1189         async with conn.execute("SELECT dedup_key FROM content_items ORDER BY
dedup_key") as cur:
1190             keys = [r[0] for r in await cur.fetchall()]
1191             assert len(keys) == 4, f"expected 4 content items, got {len(keys)}"
1192             assert len(set(keys)) == 4, f"dedup_key collision: {keys}"

```

```

1193         # Branch coverage: one tg:doc:, one tg:fp:, two tg:msg: (different
1194         # message ids ? distinct keys even with the same source_id).
1195         prefixes = {k.split(":", 2)[1] for k in keys}
1196         assert prefixes == {"doc", "fp", "msg"}
1197         assert sum(1 for k in keys if k.startswith("tg:doc:")) == 1
1198         assert sum(1 for k in keys if k.startswith("tg:fp:")) == 1
1199         assert sum(1 for k in keys if k.startswith("tg:msg:")) == 2
1200
1201         # sources backfilled with watched=1 (ADR 0009 backfill).
1202         async with conn.execute(
1203             "SELECT provider_source_id, watched FROM sources ORDER BY
provider_source_id"
1204         ) as cur:
1205             src_rows = await cur.fetchall()
1206             assert src_rows == [("-1001", 1), ("1002", 1)]
1207
1208         # policy backfilled.
1209         async with conn.execute(
1210             "SELECT name, is_default FROM policies WHERE account_id = 111"
1211         ) as cur:
1212             pol = await cur.fetchone()
1213             assert pol == ("default", 1)
1214
1215
1216     async def test_migration_003_is_idempotent(tmp_path: object) -> None:
1217         """Running migration 003 twice (via init_db) is a no-op on the second run."""
1218         db_path = str(tmp_path) + "/idem003.db" # type: ignore[operator]
1219         await db.init_db(db_path)
1220         await db.init_db(db_path) # second run
1221         async with aiosqlite.connect(db_path) as conn:
1222             async with conn.execute("PRAGMA user_version") as cur:
1223                 assert (await cur.fetchone())[0] == len(db._MIGRATIONS)
1224             # New tables present, legacy tables absent (fresh DB).
1225             assert await db._table_exists(conn, "sources")
1226             assert await db._table_exists(conn, "content_items")
1227             assert await db._table_exists(conn, "policies")
1228             assert not await db._table_exists(conn, "channels")
1229

```

8. assistant (2026-07-06T06:03:07.800Z)

Now let me focus on the ADR-conformance questions. The key ones: (1) `is\_default` UNIQUE partial index ? the ADR spec says "one per account (UNIQUE partial idx)" but the code comment in `upsert\_policy` admits it's NOT created. (2) The Tier-2 no-loss claim. (3) README dedup tier advertising. Let me check the README and grep for the index.

9. user (2026-07-06T06:03:09.392Z)

```

src\tg_video_filter\db.py:129:     is_default INTEGER NOT NULL DEFAULT 0,
src\tg_video_filter\db.py:174:# Note: the ``idx_videos_user_document`` partial unique index is
created by
src\tg_video_filter\db.py:329:     """Add ``document_id`` column + partial unique index (ADR 0006
content dedup).
src\tg_video_filter\db.py:336:     # Create the partial unique index for both fresh and migrated
DBs. Uses
src\tg_video_filter\db.py:481:     # The partial unique index for document_id dedup (created by
migration 001
src\tg_video_filter\db.py:571:     #     (Option 2A). is_default=1 so the legacy
load_filter_config shim reads
src\tg_video_filter\db.py:575:         INSERT OR IGNORE INTO policies (account_id, name,
config_json, is_default,
src\tg_video_filter\db.py:721:         is_default=bool(row["is_default"]),
src\tg_video_filter\db.py:1032:     Exactly one policy per account should carry
``is_default=True``; callers
src\tg_video_filter\db.py:1034:     one (enforced by a partial unique index landed by future

```

```

work, not in the
src\tg_video_filter\db.py:1038:         """INSERT INTO policies (account_id, name, config_json,
is_default,
src\tg_video_filter\db.py:1043:         is_default=excluded.is_default,
src\tg_video_filter\db.py:1049:         int(policy.is_default),
src\tg_video_filter\db.py:1066:         """Return the account's default policy (`is_default=1`)."""
src\tg_video_filter\db.py:1069:         "SELECT * FROM policies WHERE account_id = ? AND
is_default = 1",
src\tg_video_filter\db.py:1281:     ``filter_config_deprecated``; this reads ``policies WHERE
is_default=1``
src\tg_video_filter\db.py:1300:         is_default=True,
src\tg_video_filter\models.py:192:     Exactly one policy per account carries
``is_default=True``; the legacy
src\tg_video_filter\models.py:204:     is_default: bool = Field(
src\tg_video_filter\models.py:205:         default=False, description="Exactly one per account
(UNIQUE partial index)"

```

10. user (2026-07-06T06:03:10.584Z)

```

31:- **Never forwards the same video twice** ? three-tier content dedup
32: (message instance ? Telegram `document.id` ? metadata fingerprint) means
34: [`docs/adr/0006-content-dedup.md`](docs/adr/0006-content-dedup.md).
36: filters, delivery mode/target, and dedup state.

```

11. assistant (2026-07-06T06:03:16.267Z)

Now the key findings are emerging. Let me confirm the Tier-2 loss concretely. The critical issue: v1's `get\_video\_by\_content\_fingerprint` did a **lookup** on partial data, but more importantly ? the ADR/README claim "no dedup quality loss." Let me examine the actual Tier-2 behavior difference between v1 and the new implementation.

Let me check v1 on main to compare the Tier-2 dedup and understand what "no loss" means, and verify whether the `is\_default` partial index is created anywhere.

12. user (2026-07-06T06:03:21.194Z)

```

30:     telegram_id INTEGER UNIQUE,
51:     UNIQUE (user_id, channel_id)
59:     document_id INTEGER,
68:     UNIQUE (user_id, channel_id, message_id)
78:# migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
159:async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:
160:     """Add ``document_id`` column + partial unique index (ADR 0006 content dedup).
165:     if not await _column_exists(conn, "videos", "document_id"):
166:         await conn.execute("ALTER TABLE videos ADD COLUMN document_id INTEGER")
170:         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
171:         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
207:         telegram_id INTEGER UNIQUE,
266:         UNIQUE (user_id, channel_id)
279:         document_id INTEGER,
288:         UNIQUE (user_id, channel_id, message_id)
291:         "id, user_id, channel_id, message_id, document_id, duration_s, width, "
312:     # The partial unique index for document_id dedup (created by migration 001
316:         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
317:         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
399:         document_id=row["document_id"],
622:async def get_video_by_document_id(
623:     conn: aiosqlite.Connection, user_id: int, document_id: int
625:     """Tier-1 content dedup: same Telegram file id ? same video.
633:         WHERE user_id = ? AND document_id = ?""",
634:         (user_id, document_id),
640:async def get_video_by_content_fingerprint(
648:     """Tier-2 content dedup: metadata fingerprint match ? probably same video.
670:async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:

```



```

674:         - ``UNIQUE (user_id, channel_id, message_id)`` ? same message instance.
675:         - ``UNIQUE (user_id, document_id) WHERE document_id IS NOT NULL`` ?
676:         same file forwarded across channels (content dedup Tier 1).
680:         (user_id, channel_id, message_id, document_id, duration_s, width, height,
687:         video.document_id,
765:         "get_video_by_document_id",
766:         "get_video_by_content_fingerprint",

```

13. user (2026-07-06T06:03:21.424Z)

No matches found

14. assistant (2026-07-06T06:03:24.850Z)

Confirmed: no partial unique index on `policies(is\_default)` exists anywhere. Now let me examine the v1 Tier-2 usage to understand the no-loss claim. The key question is whether v1's fingerprint dedup was a **lookup** used by ingestion. Let me check where `get\_video\_by\_content\_fingerprint` is called and how dedup worked in v1.

15. user (2026-07-06T06:03:26.970Z)

```

async def get_video_by_content_fingerprint(
    conn: aiomysqlite.Connection,
    user_id: int,
    size_bytes: int,
    duration_s: int,
    height: int,
    width: int,
) -> VideoRecord | None:
    """Tier-2 content dedup: metadata fingerprint match ? probably same video.

```

Catches re-uploads where Telegram assigns a new `document.id` but the file bytes are identical. Uses (size_bytes, duration_s, height, width) as a composite key ? the odds of two different videos sharing all four values are negligible for this use case.

```

"""
conn.row_factory = aiomysqlite.Row
async with conn.execute(
    """SELECT * FROM videos
       WHERE user_id = ?
          AND size_bytes = ?
          AND duration_s = ?
          AND height = ?
          AND width = ?
       LIMIT 1""",
    (user_id, size_bytes, duration_s, height, width),
) as cur:
    row = await cur.fetchone()
    return _row_to_video(row) if row else None

```

```

async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
    """Insert a new video row. Raises IntegrityError on a dedup-key collision.

```

Two dedup constraints protect against duplicates:

- ``UNIQUE (user\_id, channel\_id, message\_id)`` ? same message instance.
- ``UNIQUE (user\_id, document\_id) WHERE document\_id IS NOT NULL`` ? same file forwarded across channels (content dedup Tier 1).

```

"""
cur = await conn.execute(
    """INSERT INTO videos
       (user_id, channel_id, message_id, document_id, duration_s, width, height,
        size_bytes, mime_type, matched, forwarded, seen_at)
       VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",

```

```

        (
            video.user_id,
            video.channel_id,
            video.message_id,
            video.document_id,
            video.duration_s,
            video.width,
            video.height,
            video.size_bytes,
            video.mime_type,
            int(video.matched),
            int(video.forwarded),
            video.seen_at.isoformat(),
        ),
    )
    await conn.commit()
    return video.model_copy(update={"id": cur.lastrowid})

```

```

async def mark_forwarded(conn: aiosqlite.Connection, video_id: int) -> None:
    """Record that a video's forward to Saved Messages succeeded."""
    await conn.execute("UPDATE videos SET forwarded = 1 WHERE id = ?", (video_id,))
    await conn.commit()

```

--- filter config repos -----

```

async def load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:
    """Load a user's filter config, falling back to defaults if absent."""
    conn.row_factory = aiosqlite.Row
    async with conn.execute(
        "SELECT config_json FROM filter_config WHERE user_id = ?", (user_id,)
    ) as cur:
        row = await cur.fetchone()
    if row is None:
        return FilterConfig()
    return FilterConfig.model_validate_json(row["config_json"])

```

16. user (2026-07-06T06:03:27.687Z)

```

src\tg_video_filter\backfill.py:171:             existing = await
db.get_video_by_document_id(conn, user.id, document_id)
src\tg_video_filter\backfill.py:192:             existing = await
db.get_video_by_content_fingerprint(
src\tg_video_filter\db.py:1137:async def get_video_by_document_id(
src\tg_video_filter\db.py:1151:async def get_video_by_content_fingerprint(
src\tg_video_filter\db.py:1371:     "get_video_by_document_id",
src\tg_video_filter\db.py:1372:     "get_video_by_content_fingerprint",
src\tg_video_filter\models.py:331:     def content_fingerprint(self) -> tuple[int, int, int, int]
| None:
src\tg_video_filter\telethon_client.py:193:             existing = await
db.get_video_by_document_id(conn, user.id, document_id)
src\tg_video_filter\telethon_client.py:214:             existing = await
db.get_video_by_content_fingerprint(

```

17. assistant (2026-07-06T06:03:31.981Z)

Now I understand the dedup flow. Let me look at how the ingest path uses these Tier lookups, particularly the fingerprint case, to understand what "no loss" means and whether there's a real behavioral gap. Let me read the telethon_client ingest logic.

18. user (2026-07-06T06:03:33.334Z)

```

180     channel_id = _peer_id(message.peer_id)
181     message_id = message.id
182     document_id = _extract_document_id(message)
183
184     # --- three-tier dedup (cheapest first) ---
185
186     # Tier 0: same (user, channel, message) instance ? already processed.
187     existing = await db.get_video(conn, user.id, channel_id, message_id)
188     if existing is not None:
189         return
190
191     # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
192     if document_id is not None:
193         existing = await db.get_video_by_document_id(conn, user.id, document_id)
194         if existing is not None:
195             log.info(
196                 "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
197                 "original_channel=%s original_msg=%s",
198                 user.id,
199                 document_id,
200                 channel_id,
201                 message_id,
202                 existing.channel_id,
203                 existing.message_id,
204             )
205             return
206
207     # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
208     if (
209         meta.size_bytes is not None
210         and meta.duration_s is not None
211         and meta.height is not None
212         and meta.width is not None
213     ):
214         existing = await db.get_video_by_content_fingerprint(
215             conn,
216             user.id,
217             meta.size_bytes,
218             meta.duration_s,
219             meta.height,
220             meta.width,
221         )
222         if existing is not None:
223             log.info(
224                 "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
225                 "channel=%s msg=%s original_channel=%s original_msg=%s",
226                 user.id,
227                 meta.size_bytes,
228                 meta.duration_s,
229                 meta.height,
230                 meta.width,
231                 channel_id,
232                 message_id,
233                 existing.channel_id,
234                 existing.message_id,
235             )
236             return
237
238     # Lazily record the channel (best-effort; failures are non-fatal).
239     try:
240         await db.upsert_channel(
241             conn,
242             Channel(user_id=user.id, channel_id=channel_id,
243 title=_channel_title(event)),
244         )

```

```

244         except Exception:
245             log.warning(
246                 "failed to upsert channel %s for user %s", channel_id, user.id,
exc_info=True
247             )
248
249         result = engine_holder[ENGINE_KEY].evaluate(meta)

```

19. assistant (2026-07-06T06:03:42.907Z)

Now I have the full picture. Note telethon_client.py and backfill.py are NOT in this PR's diff, so they still call the v1 shims. The shims `get\_video\_by\_document\_id` and `get\_video\_by\_content\_fingerprint` are preserved. So the ingest path still works.

Now the critical Tier-2 analysis. In v1, Tier-1 was a **DB-enforced UNIQUE index** (`UNIQUE (user\_id, document\_id)`), and Tier-2 was a **query-only lookup** (no unique constraint). Under Option 1A, `dedup\_key` collapses document_id OR fingerprint into one key with `UNIQUE(account\_id, dedup\_key)`.

The key subtle Tier-2 loss: In v1, a video with a `document\_id` set that is a **re-upload** (new document_id, same bytes) would be caught by Tier-2 fingerprint lookup. Under 1A, if `document\_id` is present, the key is `tg:doc:{document\_id}` ? the fingerprint branch is **never reached**. So a re-upload with a NEW document_id gets `tg:doc:{new\_id}`, which does NOT collide with the original's `tg:doc:{old\_id}`. Let me verify v1 catches this case that 1A misses.

20. user (2026-07-06T06:03:44.361Z)

```

184         # --- three-tier dedup (cheapest first) ---
185
186         # Tier 0: same (user, channel, message) instance ? already processed.
187         existing = await db.get_video(conn, user.id, channel_id, message_id)

```

21. assistant (2026-07-06T06:03:51.718Z)

Confirmed the v1 ingest flow: Tier 1 (document_id lookup) and Tier 2 (fingerprint lookup) are **independent, sequential checks**. A re-upload gets a NEW `document\_id`, so Tier 1 misses (new doc id not in DB), but Tier 2 catches it because the fingerprint (size/dur/h/w) matches the original.

This is exactly the case ADR 0011 Option 1C is "Rejected" for losing ("someone re-encodes and re-uploads the same video to a different channel"). But **Option 1A has the identical loss** because when `document\_id` is present, the key is `tg:doc:{id}` and the fingerprint is never consulted. Let me verify with a concrete repro against the actual shims.

22. assistant (2026-07-06T06:04:04.730Z)

```

import asyncio, sqlite3
import aiosqlite
from tg_video_filter import db
from tg_video_filter.models import Account, VideoRecord

async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    await db.upsert_account(conn, Account(id=111, telegram_id=111))

    # Original upload: doc_id=1000, fingerprint (size=5MB,dur=60,h=1080,w=1920)
    v1 = VideoRecord(user_id=111, channel_id=-1001, message_id=1,
                     document_id=1000, size_bytes=5_000_000, duration_s=60,
                     height=1080, width=1920)
    await db.insert_video(conn, v1)

    # Re-upload of the SAME video to a DIFFERENT channel: Telegram assigns a
    # NEW document_id=2000, but the bytes/metadata are identical.

```

```

v2 = VideoRecord(user_id=111, channel_id=-1002, message_id=9,
                  document_id=2000, size_bytes=5_000_000, duration_s=60,
                  height=1080, width=1920)

# --- Simulate the v1 ingest dedup sequence (Tier1 then Tier2) ---
# Tier 1: doc_id lookup for the NEW doc_id 2000
t1 = await db.get_video_by_document_id(conn, 111, 2000)
print("Tier-1 (doc_id=2000) hit?", t1 is not None) # expect False (new id)
# Tier 2: fingerprint lookup
t2 = await db.get_video_by_content_fingerprint(conn, 111, 5_000_000, 60, 1080, 1920)
print("Tier-2 (fingerprint) hit?", t2 is not None,
      "-> msg", t2.message_id if t2 else None) # v1: expect True (catches re-upload)

# --- What Option 1A's DB unique constraint actually enforces ---
# dedup_key for v2 = tg:doc:2000 (fingerprint branch never reached because
# document_id is present). Does insert_video(v2) collide?
try:
    await db.insert_video(conn, v2)
    print("insert_video(v2 re-upload) SUCCEEDED -> DUPLICATE ROW admitted")
except sqlite3.IntegrityError as e:
    print("insert_video(v2) rejected by DB:", e)

# Show dedup keys stored
conn.row_factory = aiolsqlite.Row
async with conn.execute("SELECT provider_message_id, dedup_key FROM content_items ORDER BY
id") as cur:
    for r in await cur.fetchall():
        print("  row msg=%s dedup_key=%s" % (r["provider_message_id"], r["dedup_key"]))

await conn.close()

asyncio.run(main())

```

23. user (2026-07-06T06:04:04.731Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

24. user (2026-07-06T06:04:20.974Z)

```

Tier-1 (doc_id=2000) hit? False
Tier-2 (fingerprint) hit? False -> msg None
insert_video(v2 re-upload) SUCCEEDED -> DUPLICATE ROW admitted
  row msg=1 dedup_key=tg:doc:1000
  row msg=9 dedup_key=tg:doc:2000

```

25. assistant (2026-07-06T06:04:27.595Z)

This is a striking result. Even the `**shim**` ``get_video_by_content_fingerprint`` now MISSES the re-upload. The Tier-2 fingerprint lookup, which in v1 caught re-uploads with a new document_id, now returns ``None`` because the stored ``content_items`` row has ``dedup_key='tg:doc:1000'`` (fingerprint branch not taken during ingest), so a fingerprint-based ``get_content_item_by_dedup_key`` lookup for ``tg:fp:...`` finds nothing.

Let me confirm this by comparing v1's behavior on main. Let me verify v1 catches it.

26. user (2026-07-06T06:04:38.689Z)

```
v1 Tier-2 fingerprint hit? True -> (1,)
```

27. assistant (2026-07-06T06:04:45.722Z)

Confirmed the regression precisely:

```
- **v1 (main):** Tier-2 fingerprint lookup catches the re-upload (returns the original) ? the
```

re-upload is deduped, NOT re-forwarded.

- **PR (1A):** Both the DB constraint AND the `get\_video\_by\_content\_fingerprint` shim MISS it ? the re-upload is admitted as a duplicate and would be re-forwarded.

This is a real dedup-quality loss. The ADR's "generalizes ADR 0006 **without losing any dedup quality**" (line 366-367) and README's "Never forwards the same video twice ? three-tier content dedup" are both false as implemented. This is exactly the case 1C is rejected for, yet 1A shares it whenever `document\_id` is present (the common case ? Telegram videos almost always have a document.id).

Now let me verify the `is\_default` partial index finding, and check for one more thing: the `next\_retry\_at`/other schema columns match. Let me do a final schema diff check and confirm no index exists on policies.

28. user (2026-07-06T06:06:56.090Z)

Command running in background with ID: bzp4i3szz. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bpz4i3szz.output. You will be notified when it completes. To check interim output, use Read on that file path.

29. user (2026-07-06T06:09:03.918Z)

Command running in background with ID: bs0p09vds. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bs0p09vds.output. You will be notified when it completes. To check interim output, use Read on that file path.

30. user (2026-07-06T06:09:10.561Z)

```
policies indexes: [('sqlite_autoindex_policies_1', 1)]
default policies for account 111: 2 (ADR says must be <=1)
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro_idx.py", line 29, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 195, in run
    return runner.run(main)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725, in run_until_complete
    return future.result()
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro_idx.py", line 26, in main
    p = await db.get_default_policy(conn, 111)
    ~~~~~^~~~~~
  File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 1073, in get_default_policy
    return _row_to_policy(row) if row else None
    ~~~~~^~~~~~
  File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 716, in _row_to_policy
    return Policy(
      id=row["id"],
      ...<5 lines>...
      updated_at=row["updated_at"],
    )
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line 263, in
```

```

__init__
    validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
pydantic_core._pydantic_core.ValidationError: 2 validation errors for Policy
created_at
    Input should be a valid datetime or date, input is too short [type=datetime_from_date_parsing,
input_value='t', input_type=str]
    For further information visit https://errors.pydantic.dev/2.13/v/datetime_from_date_parsing
updated_at
    Input should be a valid datetime or date, input is too short [type=datetime_from_date_parsing,
input_value='t', input_type=str]
    For further information visit https://errors.pydantic.dev/2.13/v/datetime_from_date_parsing

```

31. assistant (2026-07-06T06:09:20.389Z)

Confirmed the two findings (the trailing datetime error is just my test's dummy ``t`` value, not a code issue):

1. ``policies`` has only ``sqlite\_autoindex\_policies\_1`` = the ``UNIQUE(account\_id, name)`` index. There is NO partial unique index on ``is\_default``. Two ``is\_default=1`` rows for one account insert cleanly (result: ``default policies for account 111: 2``).

2. The ADR schema (line 141) says ``is\_default ... -- one per account (UNIQUE partial idx)``, migration step and models.py line 205 (``"Exactly one per account (UNIQUE partial index)"``) all advertise this index. It is not created. The code comment at db.py:1034 even admits it: "enforced by a partial unique index landed by future work, not in the schema."

Now let me verify the concrete failure impact of the multi-default: ``get\_default\_policy`` uses ``LIMIT``-less query returning first row ? with two defaults, which one wins is arbitrary. Let me also double check whether ``save\_filter\_config`` can actually create a second default. It uses ``name="default"`` always, so via the shim it's safe. But ``upsert\_policy`` with any other name + ``is\_default=True`` creates a second default. That's the trigger. This is a spec-conformance divergence (ADR promises the index; it's absent).

I have enough. Let me also quickly confirm the README three-tier claim location and that no other schema column diverges. Let me read the README lines around 31.

32. user (2026-07-06T06:09:21.357Z)

```

28     - size ? 2 GiB.
29     - Forwards each match to your **Saved Messages** in real time, or to a
30       **channel you own** if you set `delivery_target` (per-user).
31     - **Never forwards the same video twice** ? three-tier content dedup
32       (message instance ? Telegram `document.id` ? metadata fingerprint) means
33       a video cross-posted to 5 channels is delivered once. See
34       [docs/adr/0006-content-dedup.md](docs/adr/0006-content-dedup.md).
35     - Supports multiple users (<100) on a single host, each with its own session,
36       filters, delivery mode/target, and dedup state.
37

```

33. assistant (2026-07-06T06:09:28.815Z)

I have all findings confirmed. The schema columns/constraints otherwise match the ADR (I verified sources, content_items, policies, destinations, routing_rules, delivery_attempts all match column-for-column; watched=1 backfill present; deprecated-table retention present; empty Track-C tables present). The divergences are the missing partial index and the false no-loss/three-tier claims.

Let me compose the findings. The README.md is not in the PR diff, but the ADR text (0011) IS a changed artifact conceptually ? however the task says "Changed files ONLY: db.py, models.py, test_db.py, test_delivery.py." The ADR and README are not in the diff. Per rules, I report defects that ship in THIS PR's diff. The Tier-2 loss is a defect in db.py's implementation (the shim + constraint). The missing index is a divergence in db.py's ``\_SCHEMA`` from the accepted spec. The false claim in models.py docstring (line 205) IS in the diff. The ADR's own false no-loss claim is a design-level finding the task explicitly asks me to state.

34. user (2026-07-06T06:09:54.647Z)

Structured output provided successfully